Reconocimiento de Patrones

Versión 2026

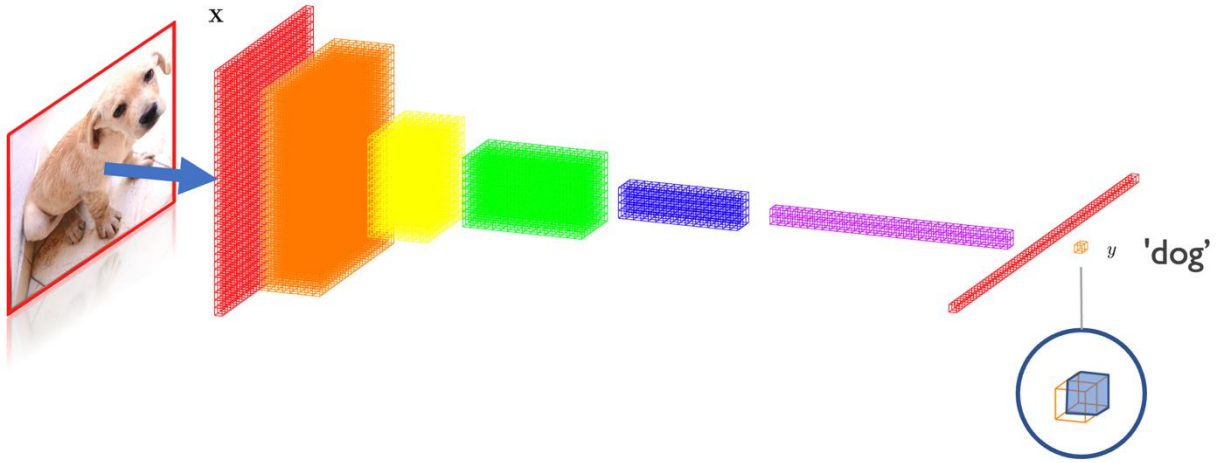


Convolutional Neural Network

(CAPÍTULO 3 – PR04E)

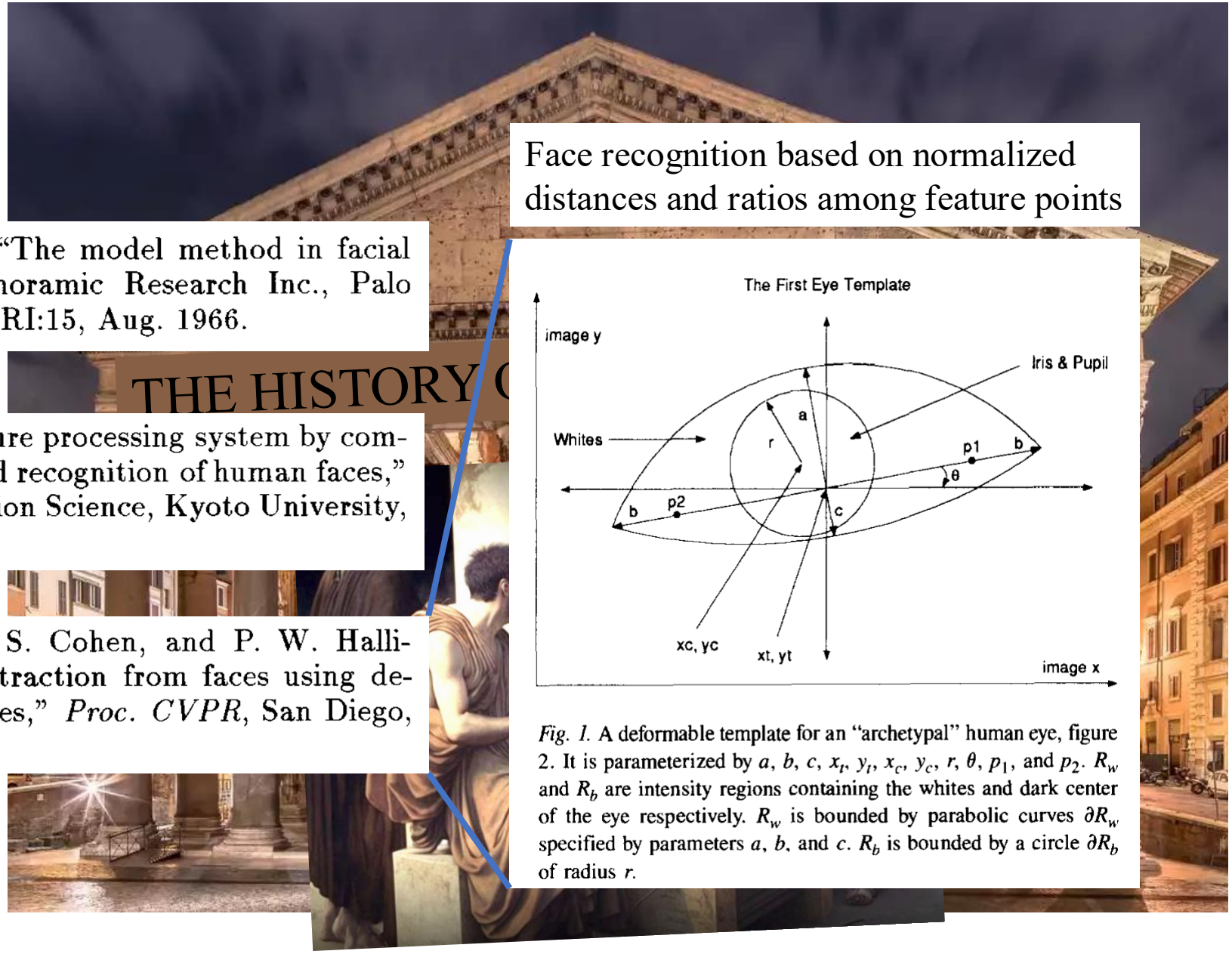
Domingo Mery

Ciencia de la Computación
Universidad Católica – Chile
http://domingomery.ing.uc.cl



Motivation


THE HISTORY OF COMPUTER VISION



Face recognition based on normalized distances and ratios among feature points

W. W. Bledsoe, "The model method in facial recognition," Panoramic Research Inc., Palo Alto, CA, Rep. PRI:15, Aug. 1966.

THE HISTORY

T. Kanade, "Picture processing system by computer complex and recognition of human faces," Dept. of Information Science, Kyoto University, Nov. 1973.

A. L. Yuille, D. S. Cohen, and P. W. Hallinan, "Feature extraction from faces using deformable templates," *Proc. CVPR*, San Diego, CA, June 1989.

Fig. 1. A deformable template for an "archetypal" human eye, figure 2. It is parameterized by a, b, c, x_t, y_t, x_c, y_c, r, θ, p_1, and p_2. R_w and R_b are intensity regions containing the whites and dark center of the eye respectively. R_w is bounded by parabolic curves ∂R_w specified by parameters a, b, and c. R_b is bounded by a circle ∂R_b of radius r.

CVPR 1991 Best Paper Award

Face Recognition Using Eigenfaces

Matthew A. Turk and Alex P. Pentland

Vision and Modeling Group, The Media Laboratory
Massachusetts Institute of Technology

Abstract

We present an approach to the detection and identification of human faces and describe a working, near-real-time face recognition system which tracks a subject's head and then recognizes the person by comparing characteristics of the face to those of known individuals. Our approach treats face recognition as a two-dimensional recognition problem, taking advantage of the fact that faces are are normally upright and thus may be described by a small set of 2-D characteristic views. Face images are projected onto a feature space ("face space") that best encodes the variation among known face images. The face space is defined by the "eigenfaces", which are the eigenvectors of the set of faces; they do not necessarily correspond to isolated features such as eyes, ears, and noses. The framework

another. The approach transforms face images into a small set of characteristic feature images, called "eigenfaces", which are the principal components of the initial training set of face images. Recognition is performed by projecting a new image into the subspace spanned by the eigenfaces ("face space") and then classifying the face by comparing its position in face space with the positions of known individuals.

Automatically learning and later recognizing new faces is practical within this framework. Recognition under reasonably varying conditions is achieved by training on a limited number of characteristic views (e.g., a "straight on" view, a 45° view, and a profile view). The approach has advantages over other face recognition schemes in its speed and simplicity, learning capacity, and relative insensitivity to small or gradual changes in the face image.

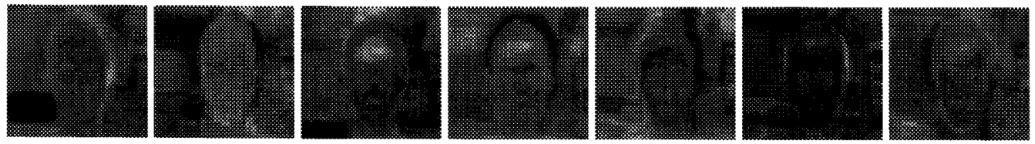
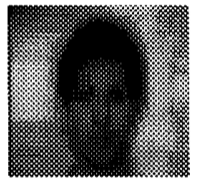


Figure 2: Seven of the eigenfaces calculated from the images of Figure 1, without the background removed.



(a)



(b)

Figure 1: (a) Face images used as the training set. (b) The average face Ψ.

ECCV 2004

Face Recognition with Local Binary Patterns

Timo Ahonen, Abdenour Hadid, and Matti Pietikäinen

Machine Vision Group, Infotech Oulu
PO Box 4500, FIN-90014 University of Oulu, Finland,
{tahonen,hadid,mkp}@ee.oulu.fi, http://www.ee.oulu.fi/mvg/

Abstract. In this work, we present a novel approach to face recognition which considers both shape and texture information to represent face images. The face area is first divided into small regions from which Local Binary Pattern (LBP) histograms are extracted and concatenated into a single, spatially enhanced feature histogram efficiently representing the face image. The recognition is performed using a nearest neighbour classifier in the computed feature space with Chi square as a dissimilarity measure. Extensive experiments clearly show the superiority of the proposed scheme over all considered methods (PCA, Bayesian Intra/extrapersonal Classifier and Elastic Bunch Graph Matching) on FERET tests which include testing the robustness of the method against different facial expressions, lighting and aging of the subjects. In addition to its efficiency, the simplicity of the proposed method allows for very fast feature extraction.

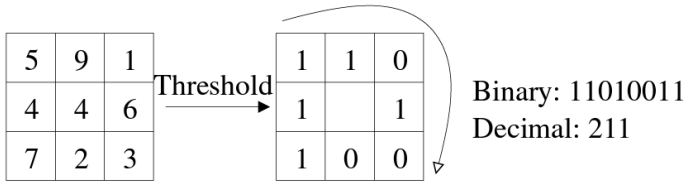
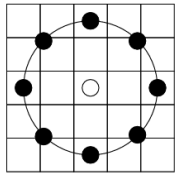


Fig. 1. The basic LBP operator.





CVPR 2014

DeepFace: Closing the Gap to Human-Level Performance in Face Verification

Yaniv Taigman Ming Yang Marc'Aurelio Ranzato Lior Wolf

Facebook AI Research
Menlo Park, CA, USA
{yaniv, mingyang, ranzato}@fb.com

Tel Aviv University
Tel Aviv, Israel
wolf@cs.tau.ac.il

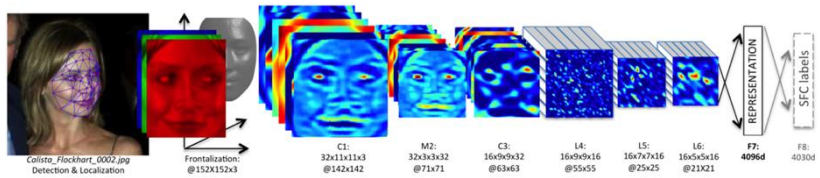


Figure 2. **Outline of the *DeepFace* architecture**. A front-end of a single convolution-pooling-convolution filtering on the rectified input, followed by three locally-connected layers and two fully-connected layers. Colors illustrate feature maps produced at each layer. The net includes more than 120 million parameters, where more than 95% come from the local and fully connected layers.

BMVC 2015

Deep Face Recognition

Omkar M. Parkhi
omkar@robots.ox.ac.uk
Andrea Vedaldi
vedaldi@robots.ox.ac.uk
Andrew Zisserman
az@robots.ox.ac.uk

Visual Geometry Group
Department of Engineering Science
University of Oxford

Abstract

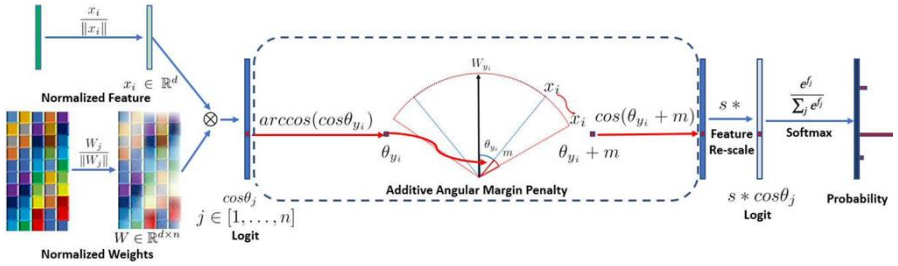
The goal of this paper is face recognition – from either a single photograph or from a set of faces tracked in a video. Recent progress in this area has been due to two factors: (i) end to end learning for the task using a convolutional neural network (CNN), and (ii) the availability of very large scale training datasets.

We make two contributions: first, we show how a very large scale dataset (2.6M images, over 2.6K people) can be assembled by a combination of automation and human in the loop, and discuss the trade off between data purity and time; second, we traverse through the complexities of deep network training and face recognition to present methods and procedures to achieve comparable state of the art results on the standard LFW and YTF face benchmarks.

CVPR 2019

ArcFace: Additive Angular Margin Loss for Deep Face Recognition

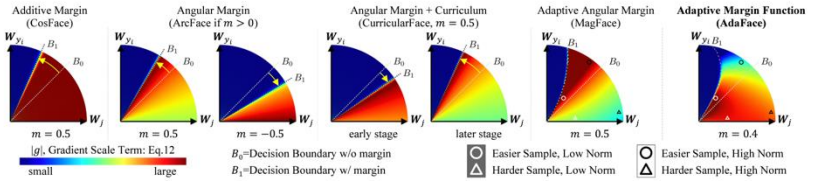
Jiankang Deng * [1,2,3] Jia Guo * [2] Niannan Xue[1] Stefanos Zafeiriou[1,3]
[1]Imperial College London [2]InsightFace [3]FaceSoft
{j.deng16, n.xue15, s.zafeiriou}@imperial.ac.uk , guojia@gmail.com



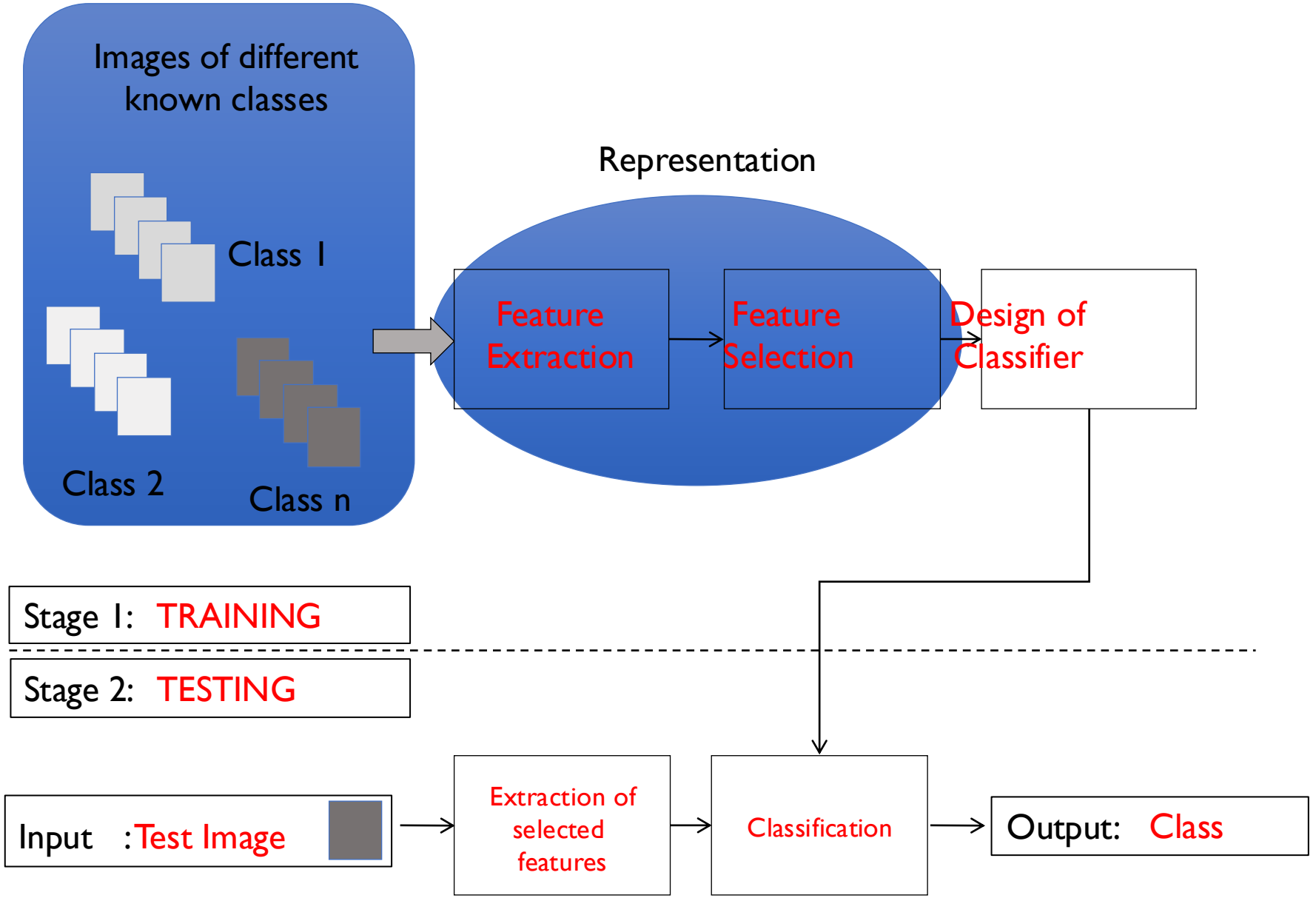
CVPR 2022

AdaFace: Quality Adaptive Margin for Face Recognition

Minchul Kim, Anil K. Jain, Xiaoming Liu
Department of Computer Science and Engineering,
Michigan State University, East Lansing, MI, 48824
{kimminc2, jain, liuxm}@cse.msu.edu



etc…


Images of different known classes
Class 1
Class 2
Class n
Representation
Feature Extraction
Feature Selection
Design of Classifier
Stage 1: TRAINING
Stage 2: TESTING
Input : Test Image
Extraction of selected features
Classification
Output: Class

MOTIVATION

- Handcrafted vs. Learned features

LBP, Haralick, Fourier, Gabor, etc.

or

Learned representation from training data

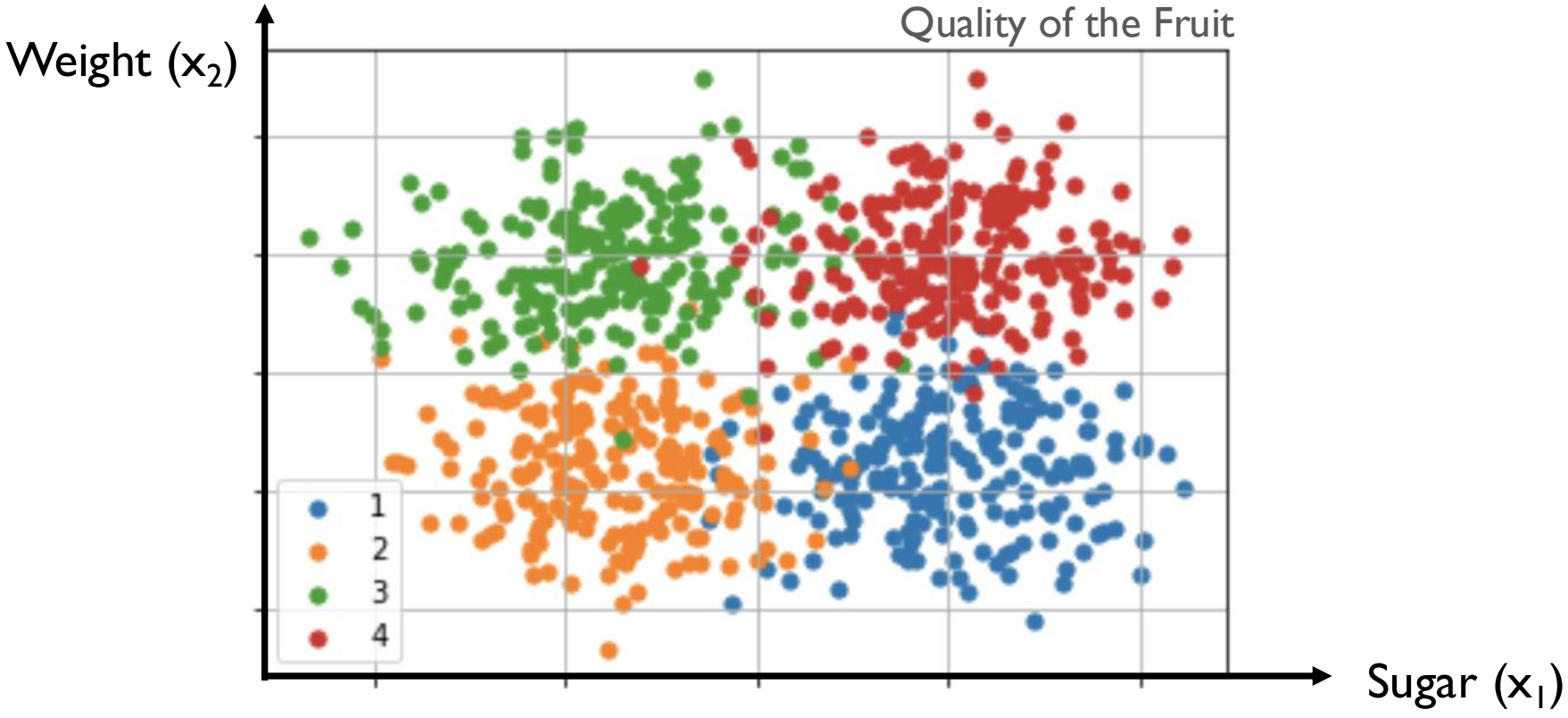
Pattern Recognition

The use of computer algorithms that take data as input and return a class, a measurement, an interpretation or a decision.

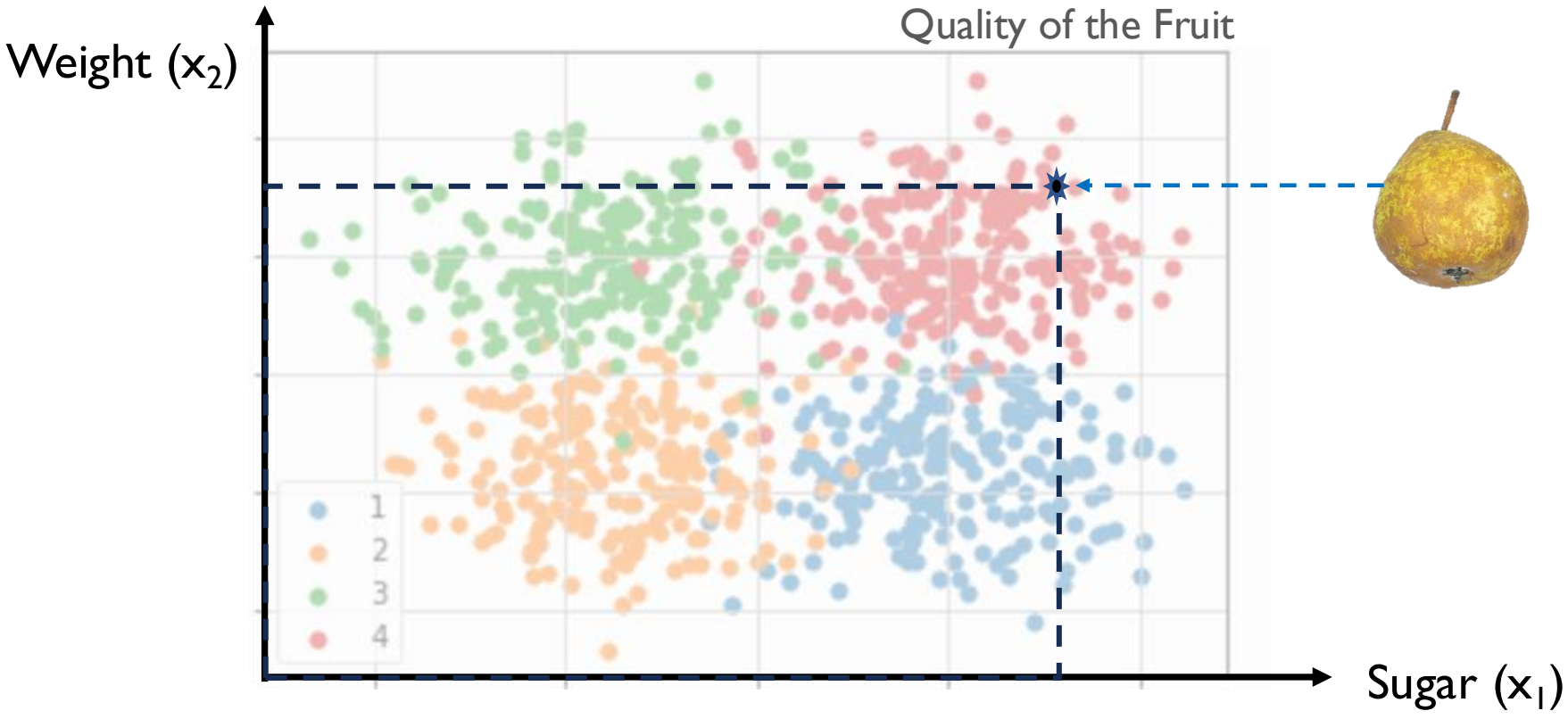


Quality: "Premium"

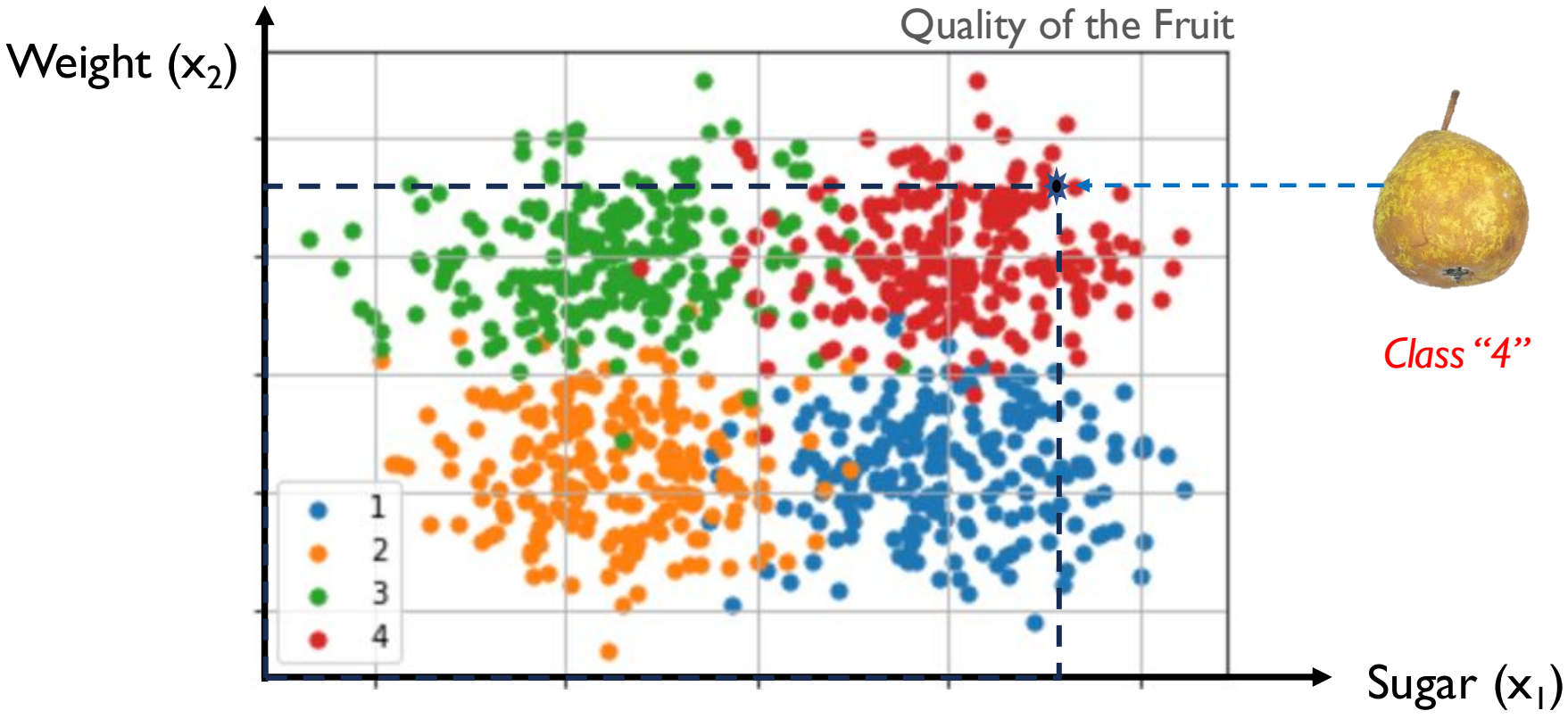
Pattern Recognition



Pattern Recognition



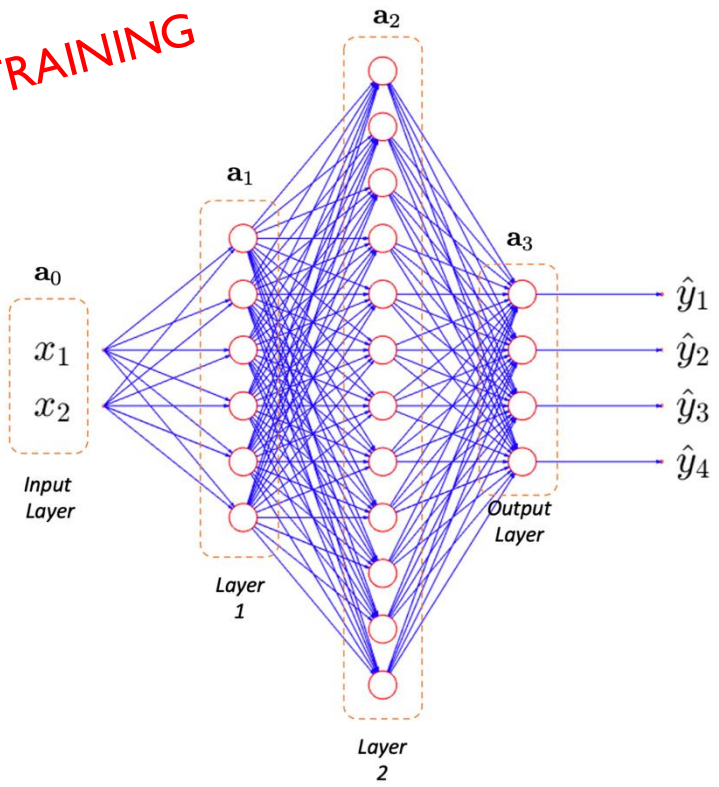
Pattern Recognition



Neural Network

errors

TRAINING



x_1	x_2
7.8402	1.5190
9.7145	0.0424
7.1526	-4.1852
13.0419	-3.3597
10.2582	4.4638
11.6146	0.5340
7.4023	-1.8662
5.8033	-3.7628
8.9302	0.9052
8.1835	3.4355
10.6855	2.7405
9.1418	-0.4821
9.9633	6.2377
10.4738	4.9295
9.3004	3.5021
9.7183	1.9630
15.1194	2.0790
4.2210	3.2440

y_1	y_2	y_3	y_4
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
1	0	0	0
0	1	0	0



Back-propagation

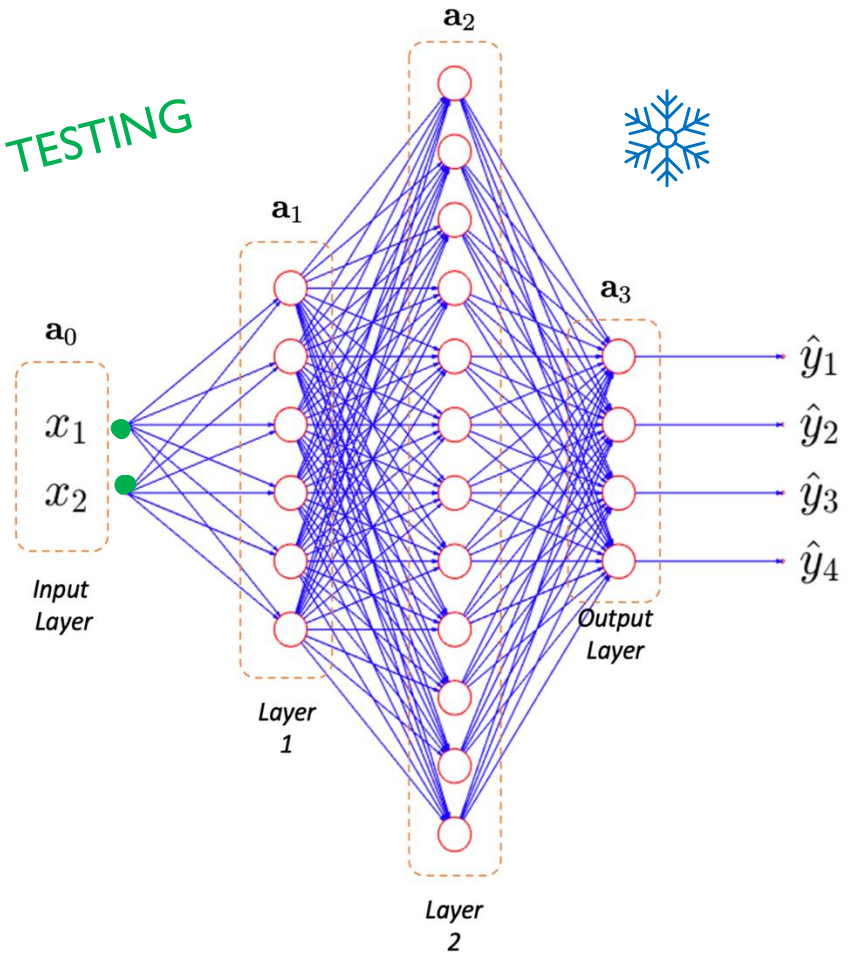
Neural Network



$x_1 = 10$

$x_2 = 7$

TESTING

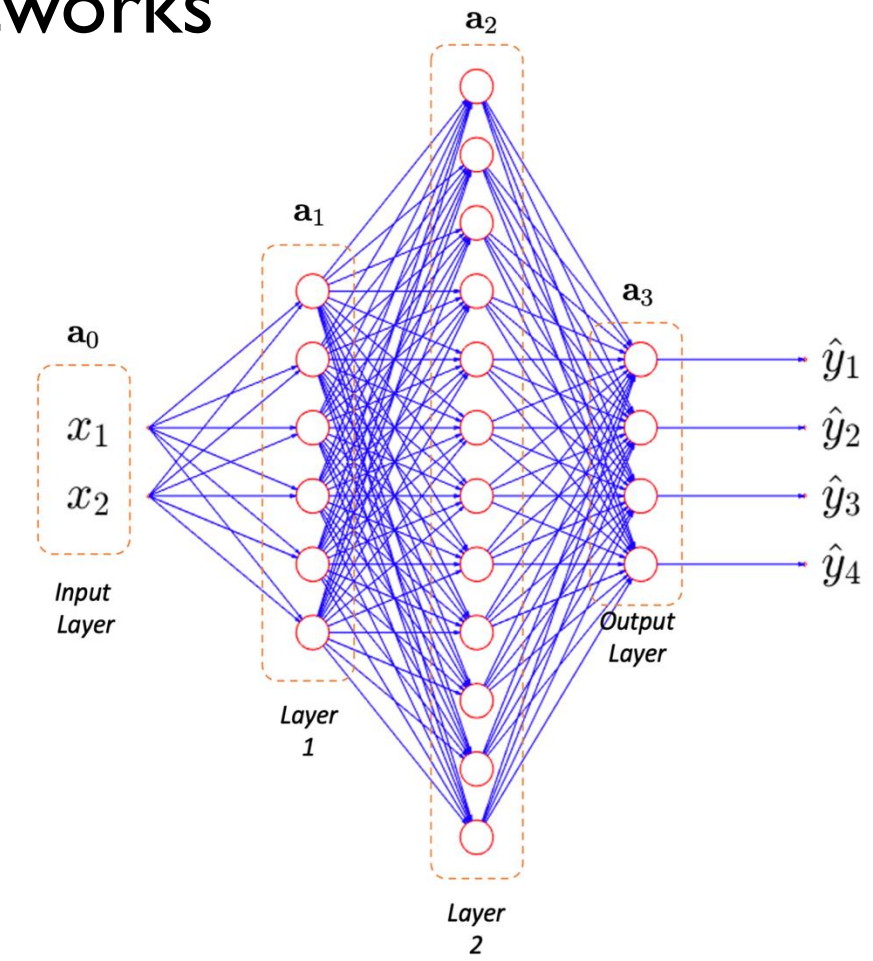


0.05
0.15
0.10
0.70

Class "4"

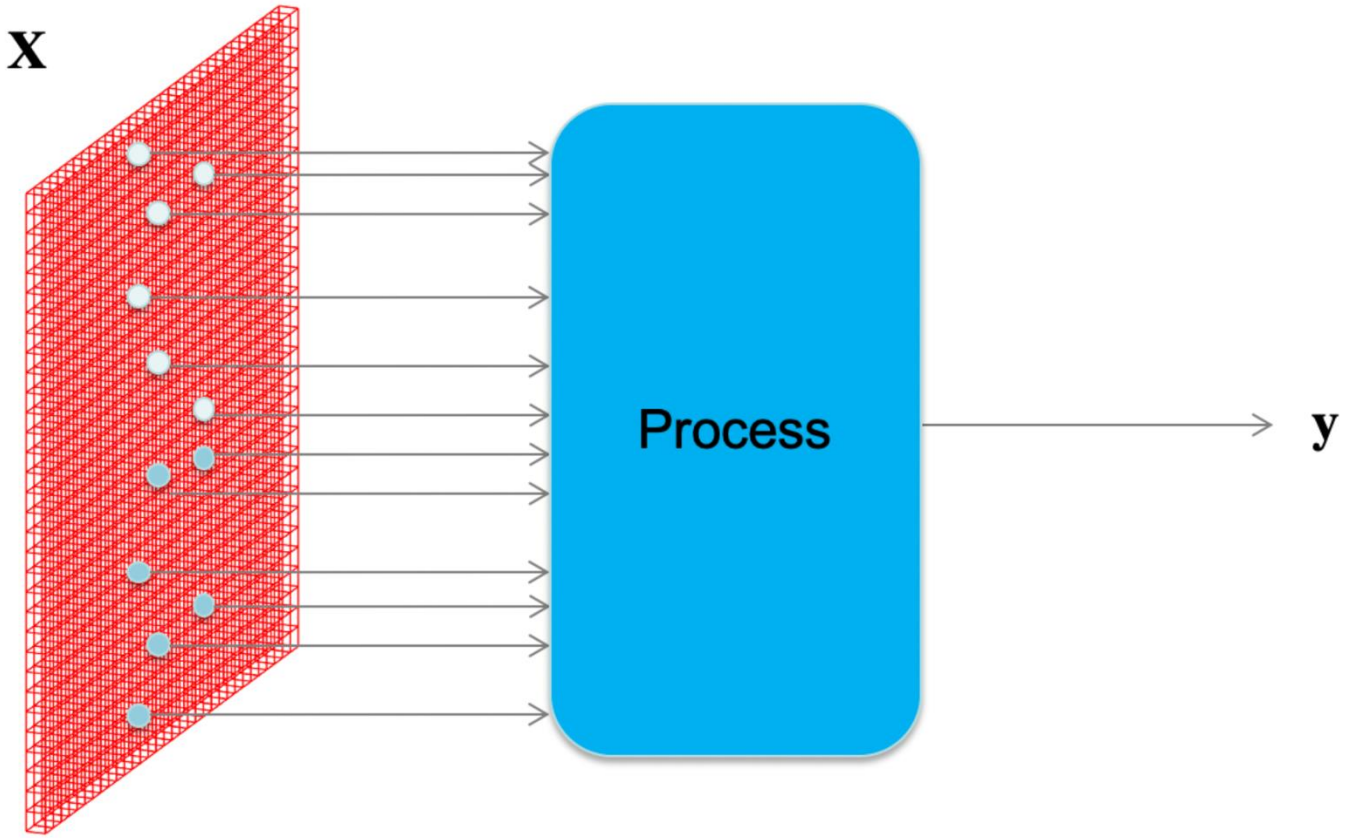
MOTIVATION

- Neural networks



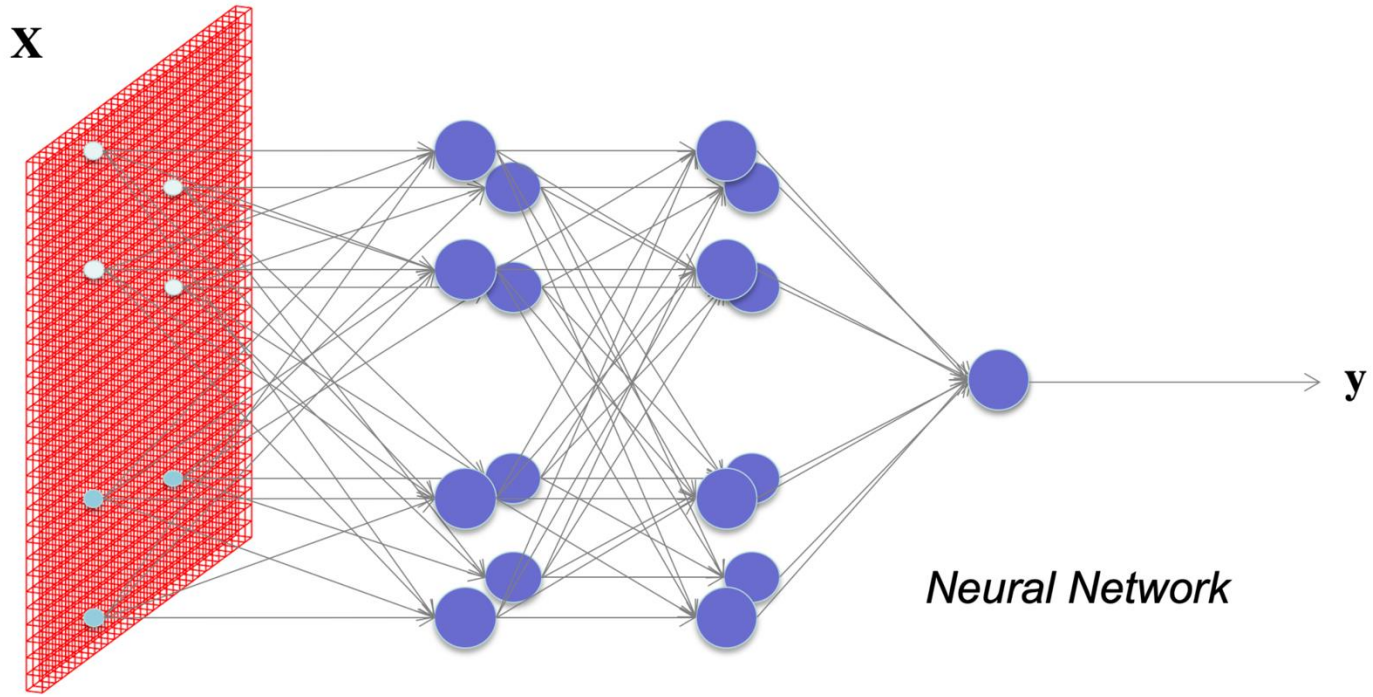
MOTIVATION

- How to process an image with neural networks?



MOTIVATION

- What happens if each pixel is an input?



How many neurons and weights?

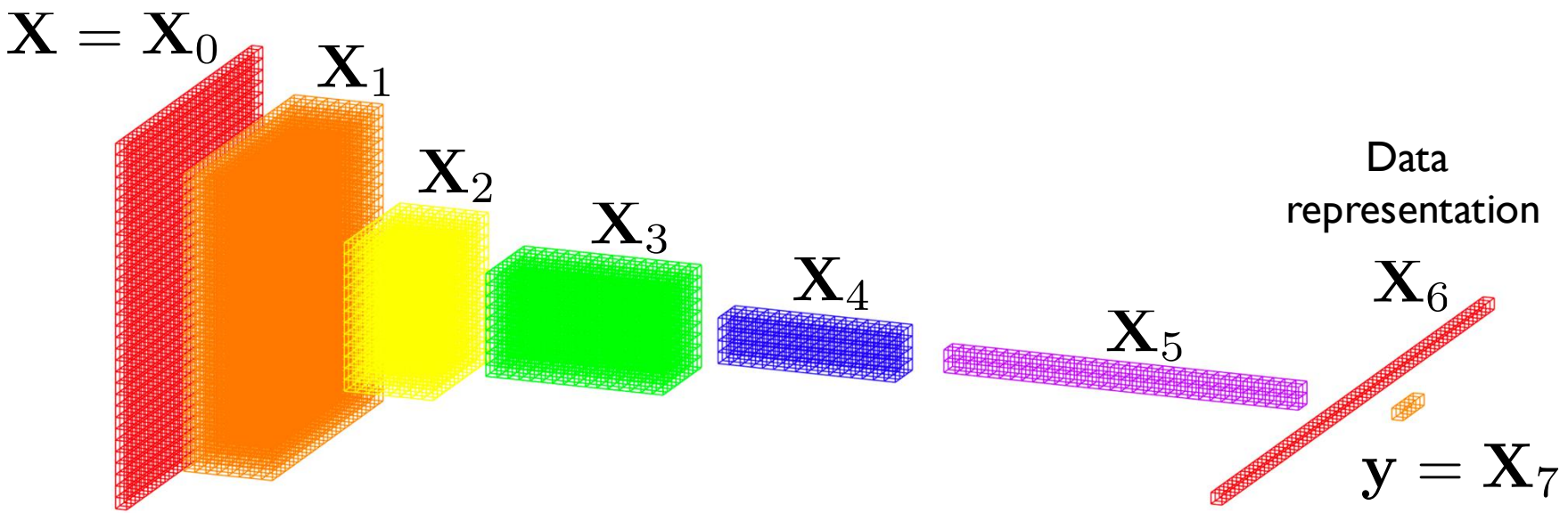


© cibercontrols.org

Basics

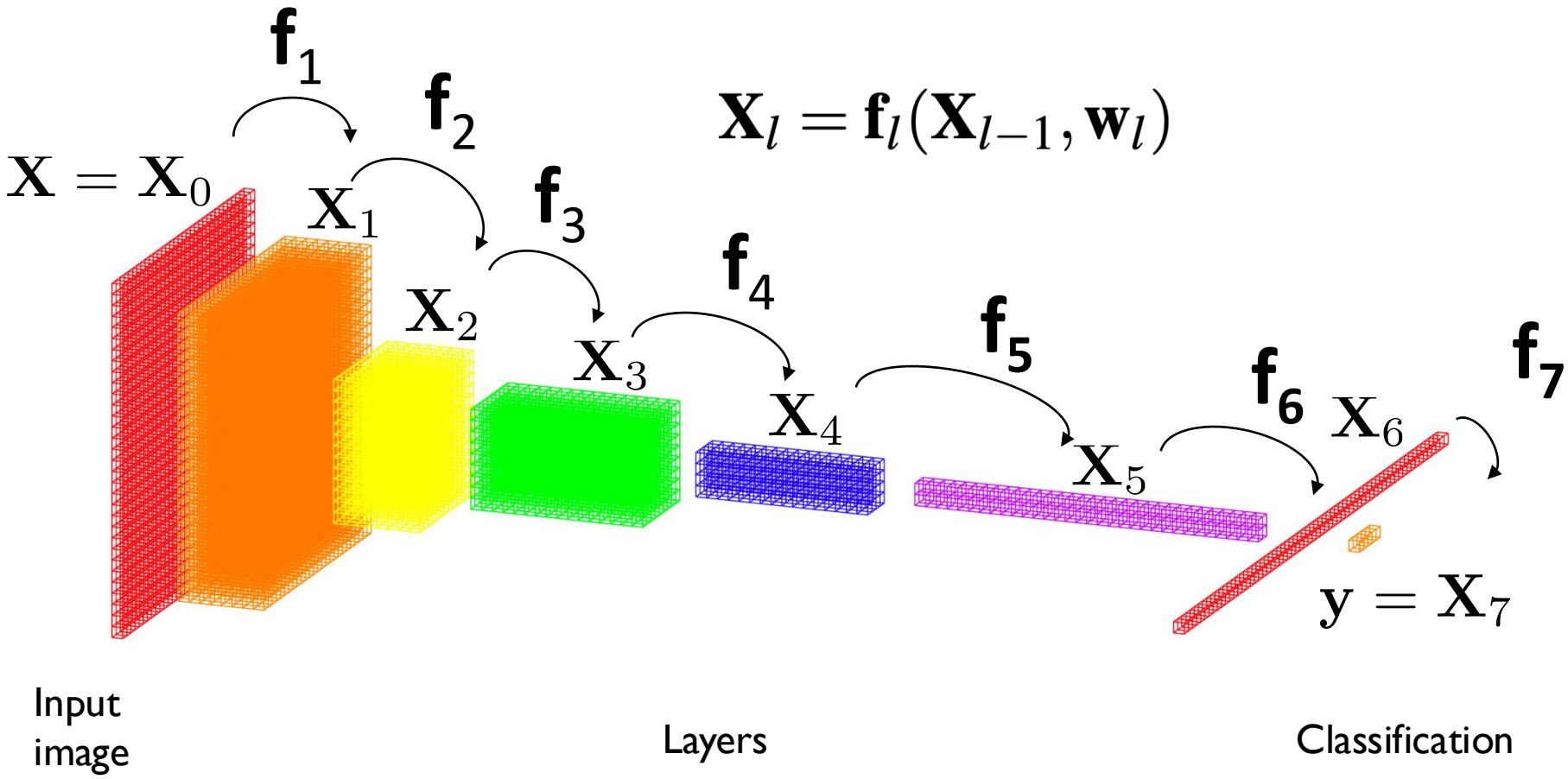
DEEP LEARNING BASICS (CNN)

Convolutional Neural Networks



DEEP LEARNING BASICS (CNN)

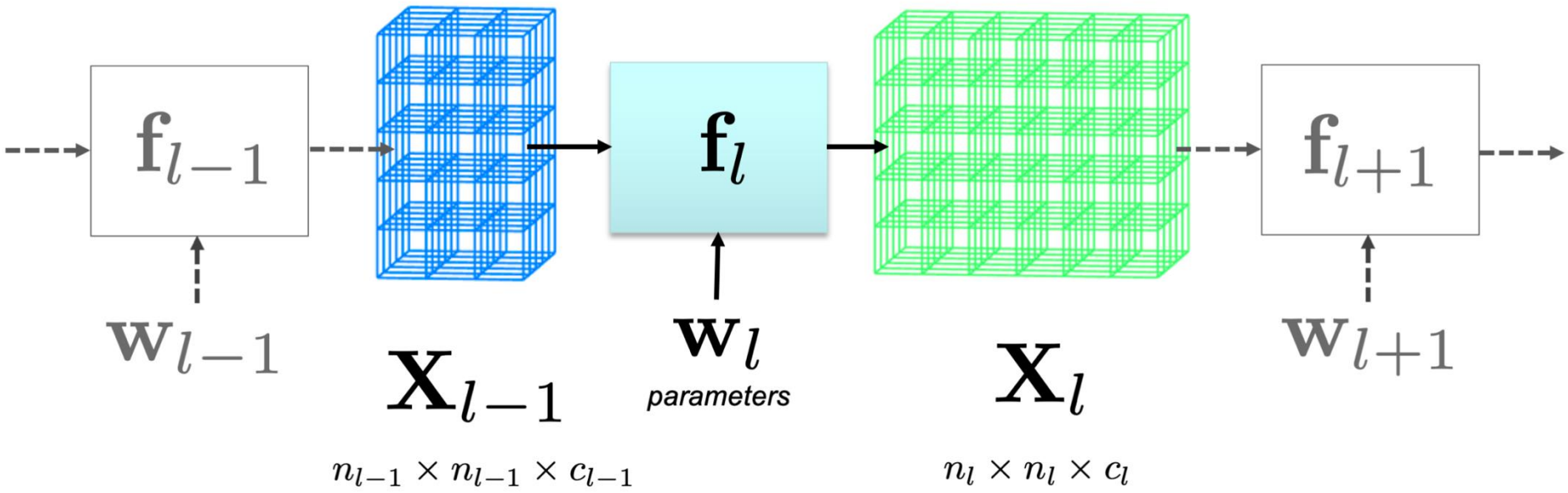
Convolutional Neural Networks



DEEP LEARNING BASICS

The convolutional neural network (CNN) can be understood as a set of layers, in which each layer processes an input image $\mathbf{X}$ in order to obtain an output image $\mathbf{Y}$:

$$\mathbf{X}_l = \mathbf{f}_l(\mathbf{X}_{l-1}, \mathbf{w}_l) \tag{1}$$



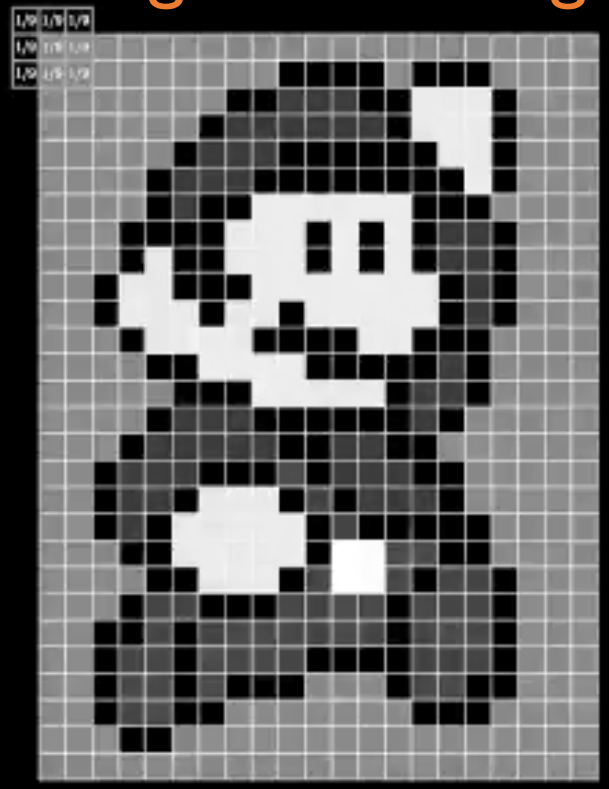
DEEP LEARNING: Typical layers

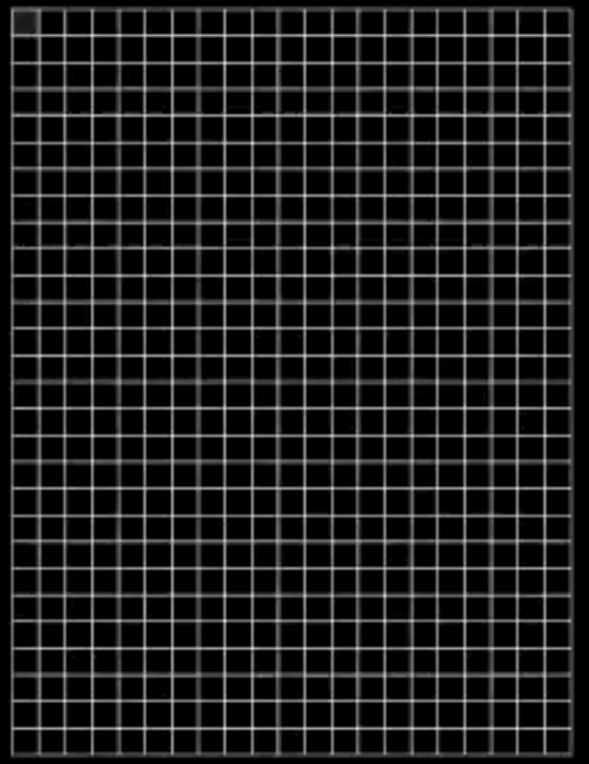
- Convolution layer
- Pooling layer
- Rectified Linear Unit
- Fully connected and SoftMax

DEEP LEARNING: Typical layers

- Convolution layer
- Pooling layer
- Rectified Linear Unit
- Fully connected and SoftMax

Image Processing: Filtering using Convolution





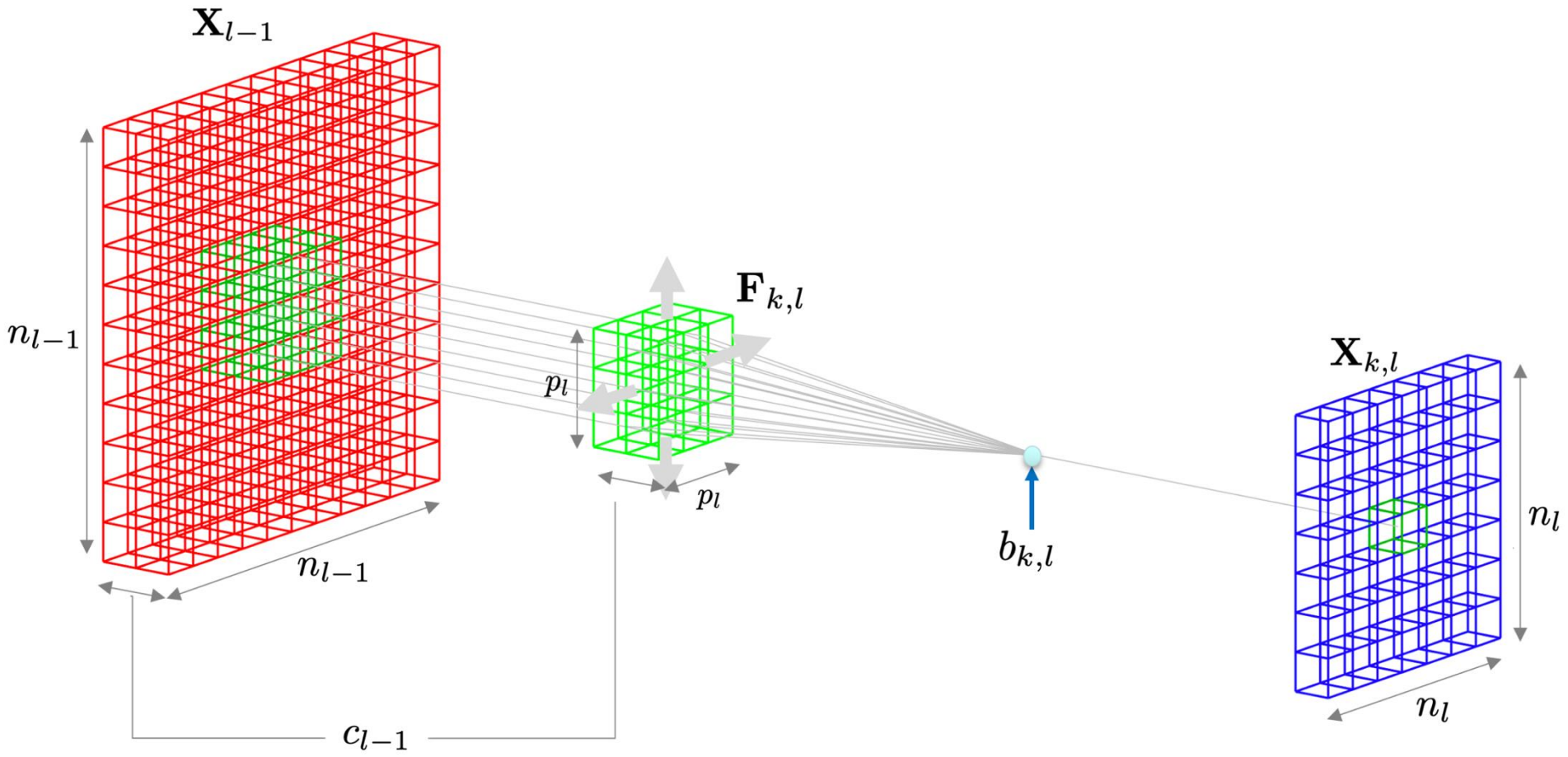
© @3blue1brown

DEEP LEARNING: Convolution layer

$$\mathbf{X}_{k,l} = \mathbf{X}_{l-1} * \mathbf{F}_{k,l} + b_{k,l} \quad \text{for } k = 1 \ldots m_l$$

$$X_{k,l}(i,j) = b_{k,l} + \sum_{u=1}^{p_l} \sum_{v=1}^{p_l} \sum_{w=1}^{q_l} X_{w,l-1}(i+u, j+v) F_{k,l}(u,v,w)$$

DEEP LEARNING: Convolution layer



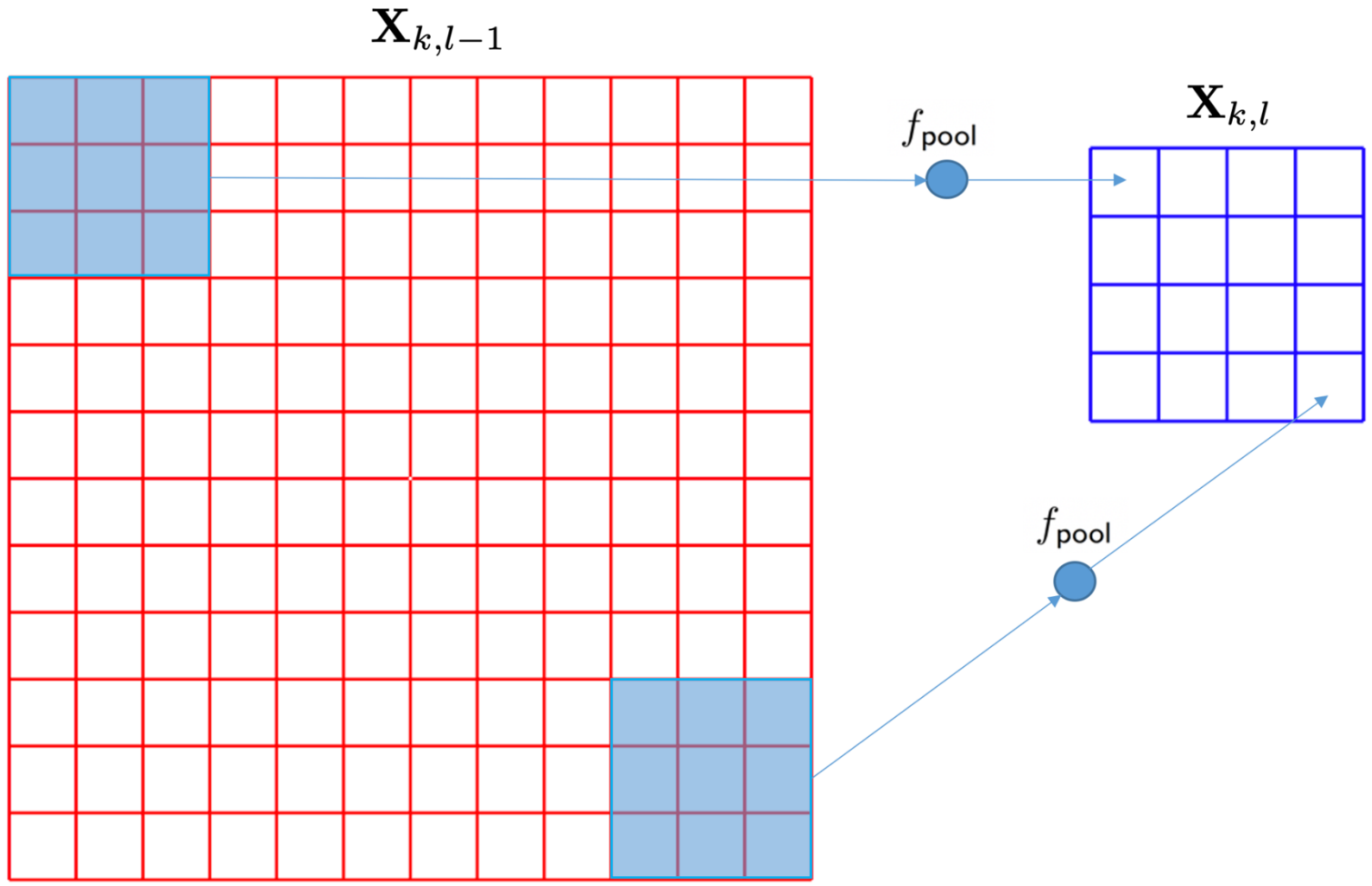
DEEP LEARNING: Typical layers

- Convolution layer
- Pooling layer
- Rectified Linear Unit
- Fully connected and SoftMax

DEEP LEARNING: Pooling layer

$$X_{k,l}(i,j) = f_{\text{pool}}\{X_{k,l-1}(u,v) : (u,v) \in \Omega(i,j)\}$$

DEEP LEARNING: Pooling layer



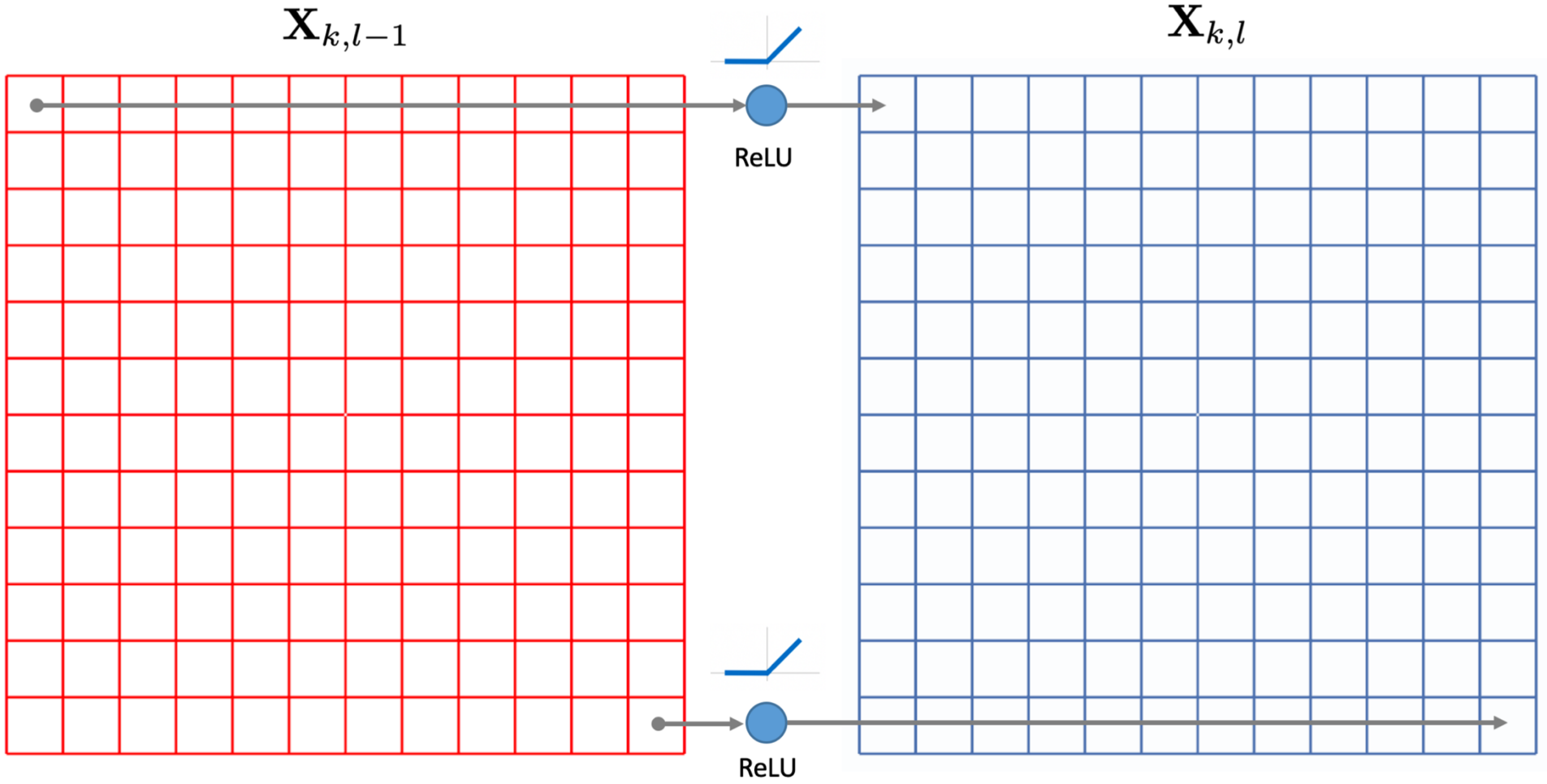
DEEP LEARNING: Typical layers

- Convolution layer
- Pooling layer
- Rectified Linear Unit
- Fully connected and SoftMax

DEEP LEARNING: Rectified Linear Unit (ReLU)

$$X_{k,l}(i,j) = \max\{0, X_{k,l-1}(i,j)\}$$

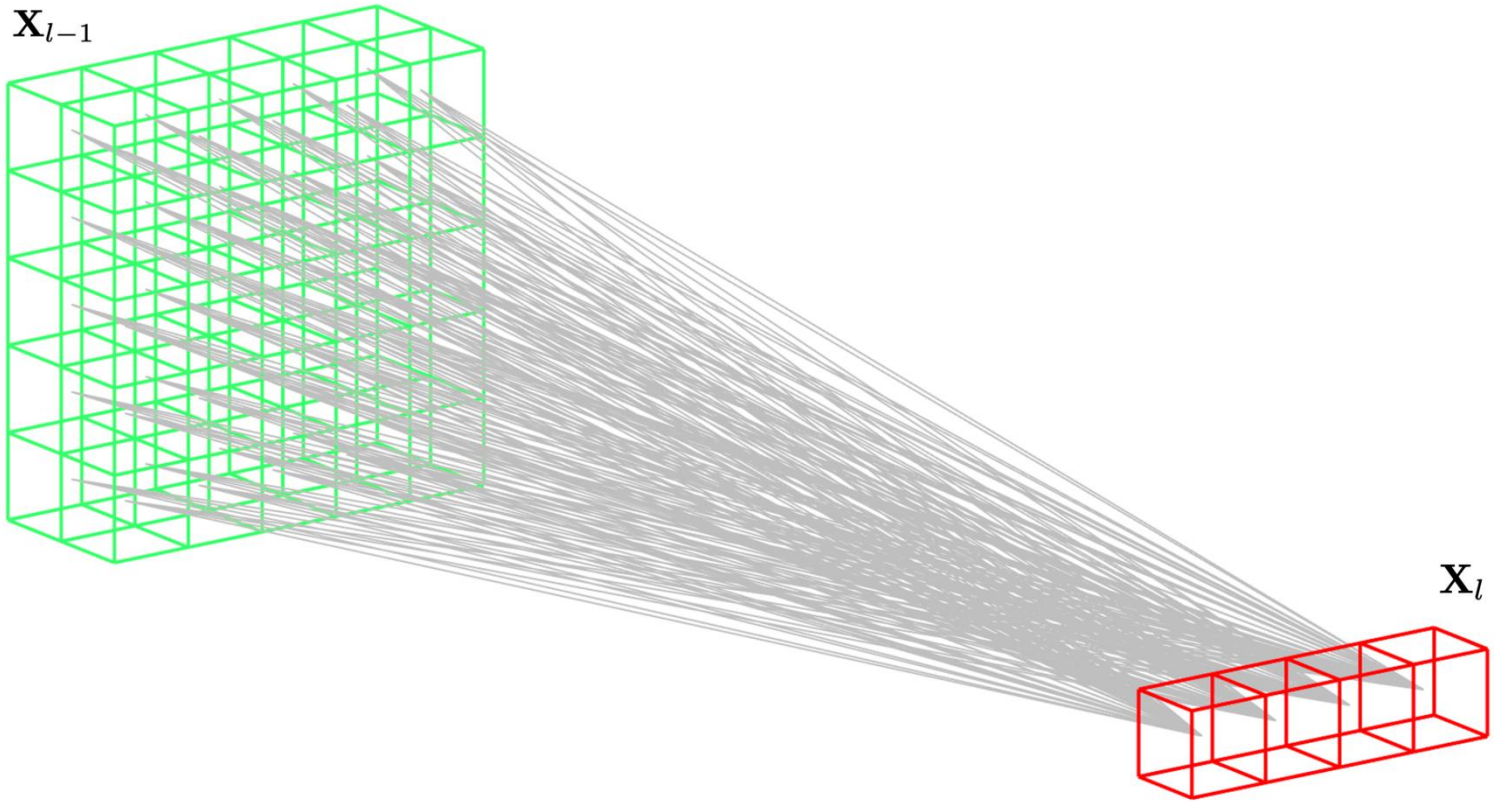
DEEP LEARNING: Rectified Linear Unit (ReLU)



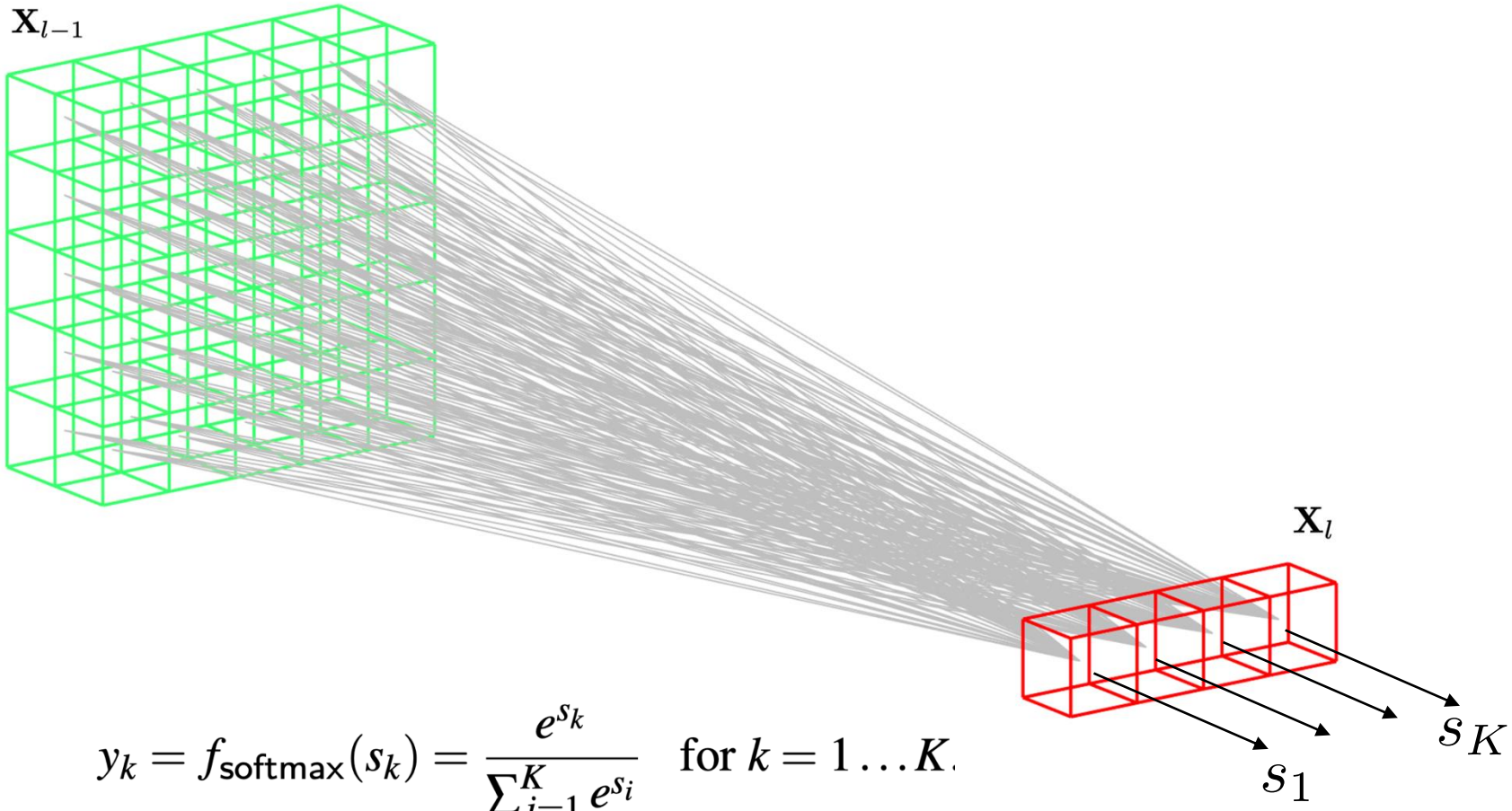
DEEP LEARNING: Typical layers

- Convolution layer
- Pooling layer
- Rectified Linear Unit
- Fully connected and SoftMax

DEEP LEARNING: Fully Connected



DEEP LEARNING: Fully Connected + SoftMax



$$y_k = f_{\text{softmax}}(s_k) = \frac{e^{s_k}}{\sum_{j=1}^{K} e^{s_i}} \quad \text{for } k = 1 \ldots K.$$

© cibercontrols.org

Training

Image Classification – Problem

> Assign an image to a class or category

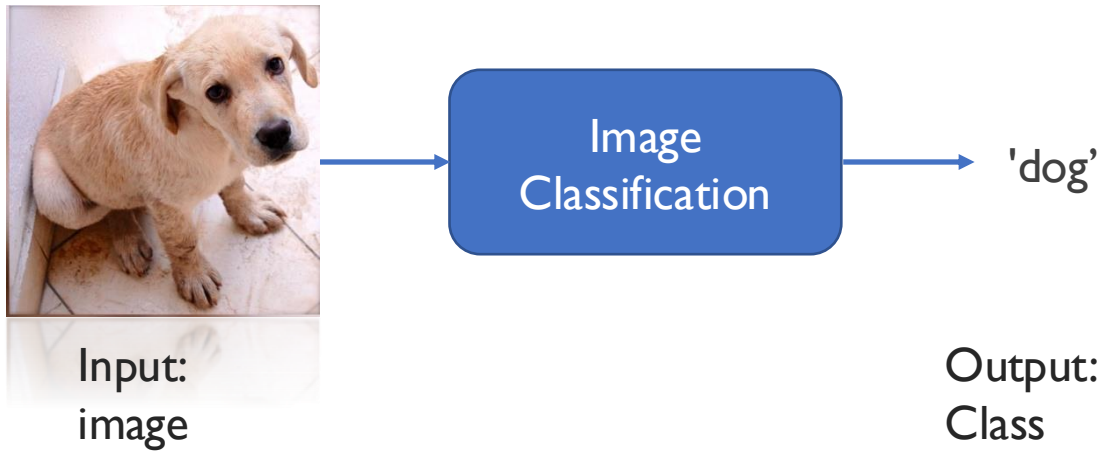


Image Classification – Problem

> Assign an image to a class or category

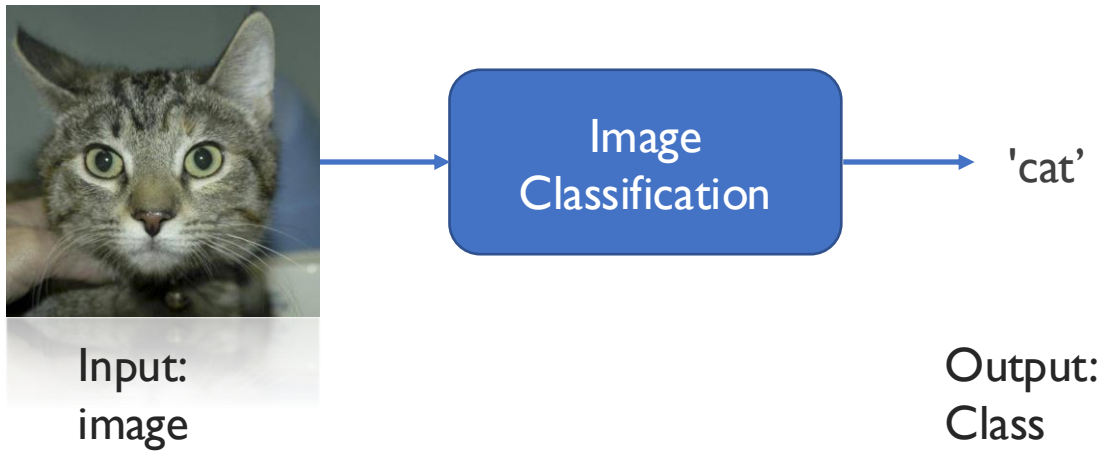


Image Classification – Solution

Deep Learning Model: CNN

Convolutional Neural Network

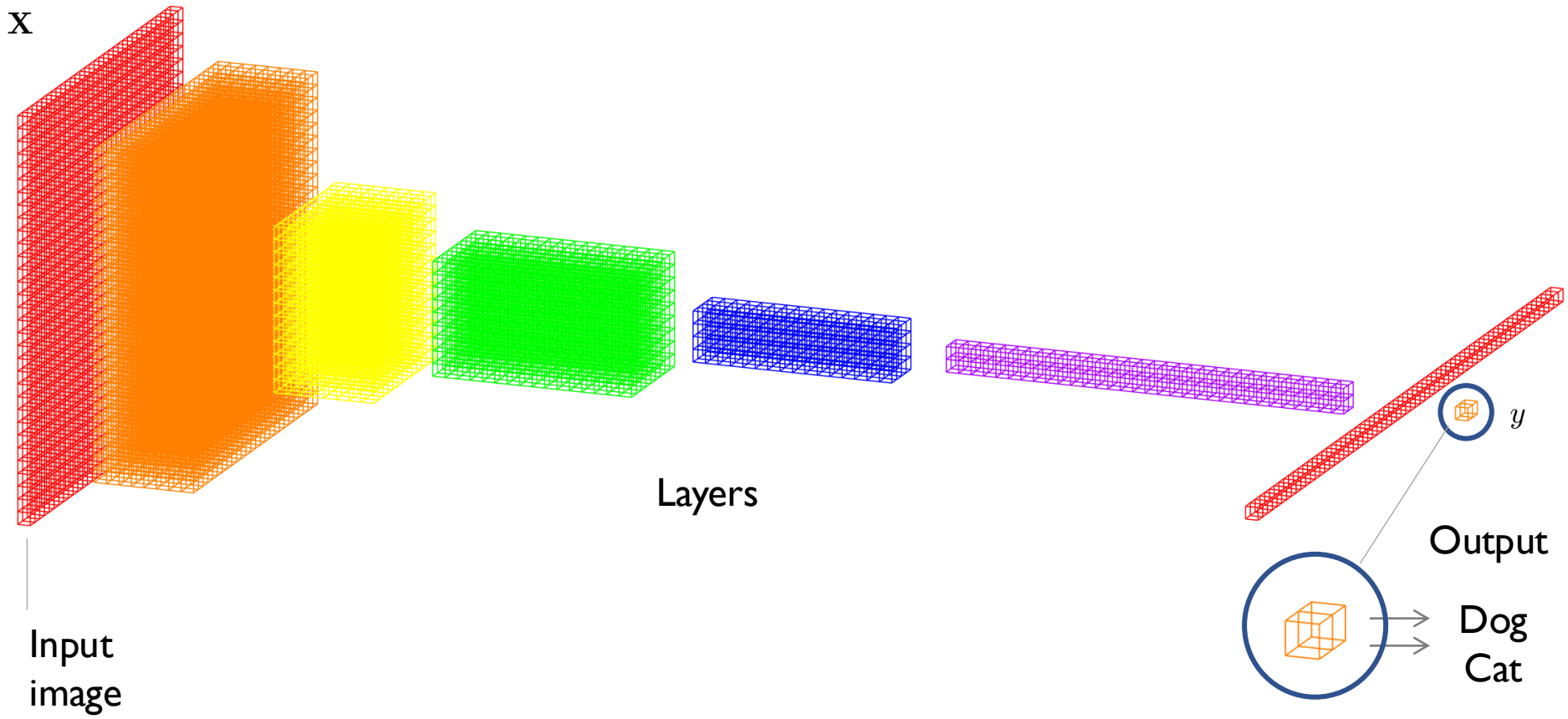
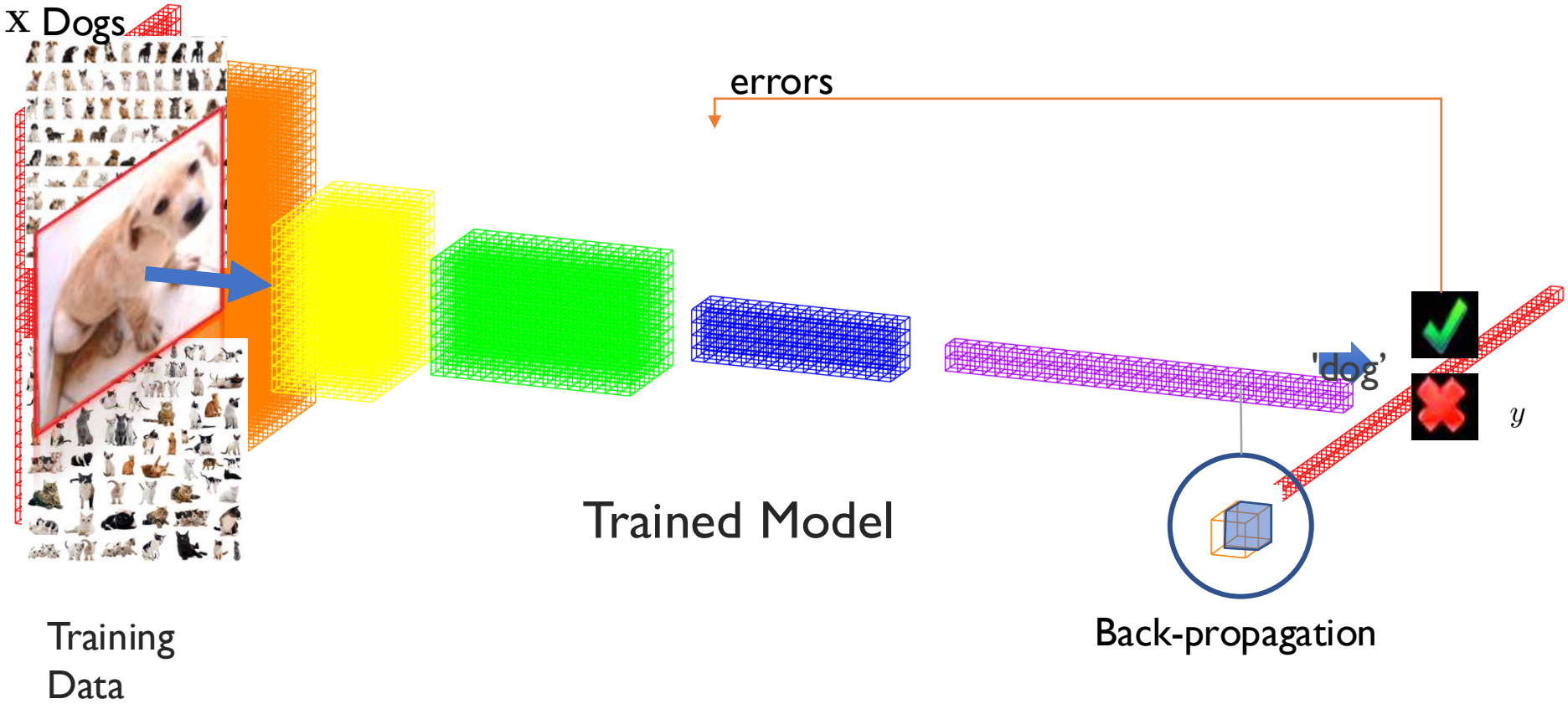


Image Classification – Solution

Deep Learning Model: CNN
Convolutional Neural Network



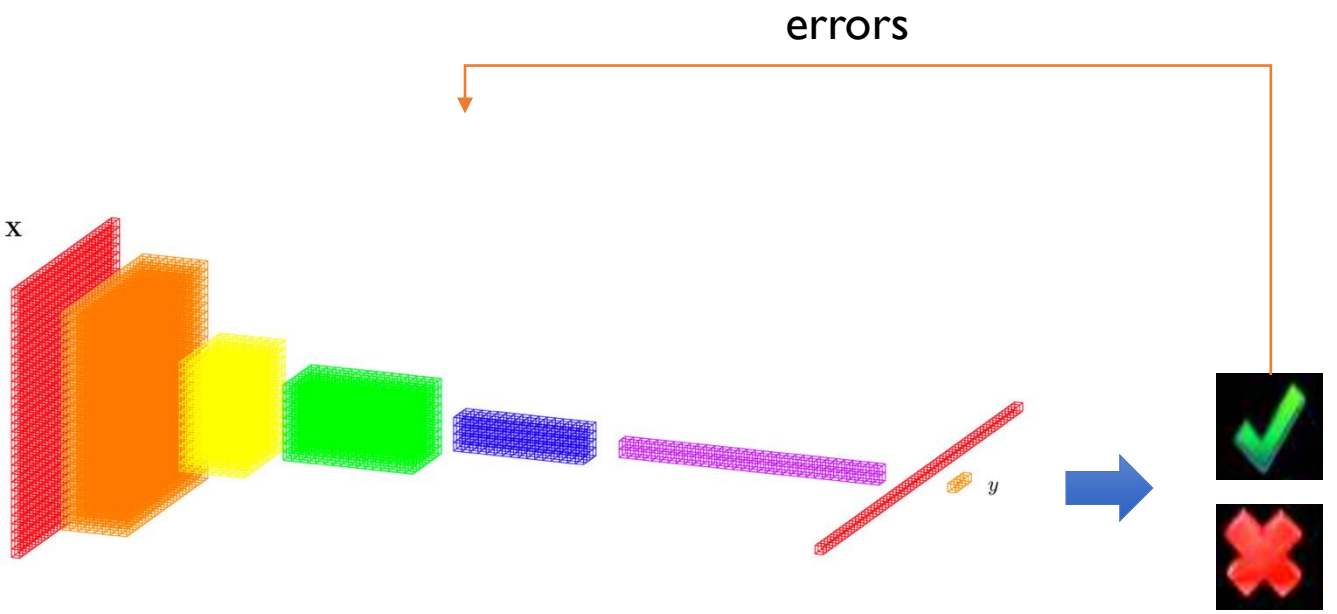
DEEP LEARNING: Training

dogs



cats

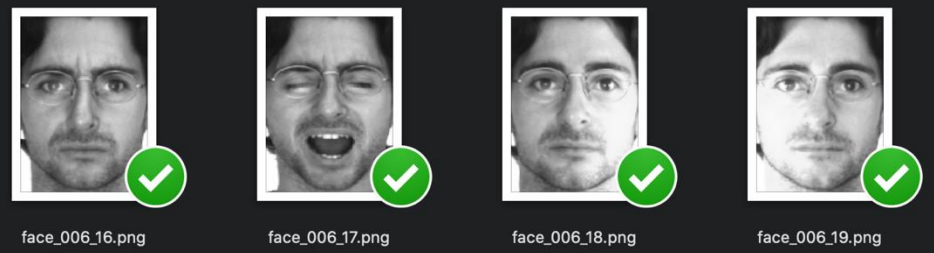


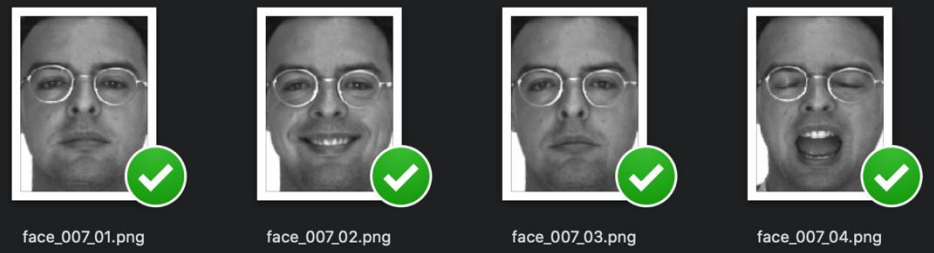


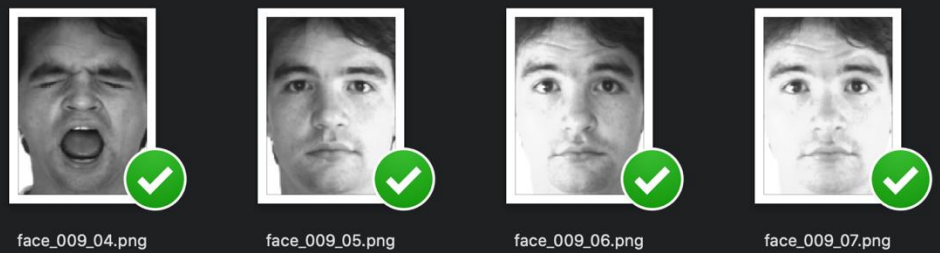
Back-propagation

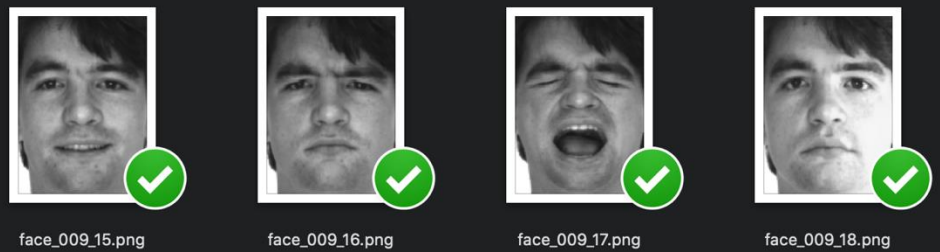
Example

Eyes vs. Nose Recognizer

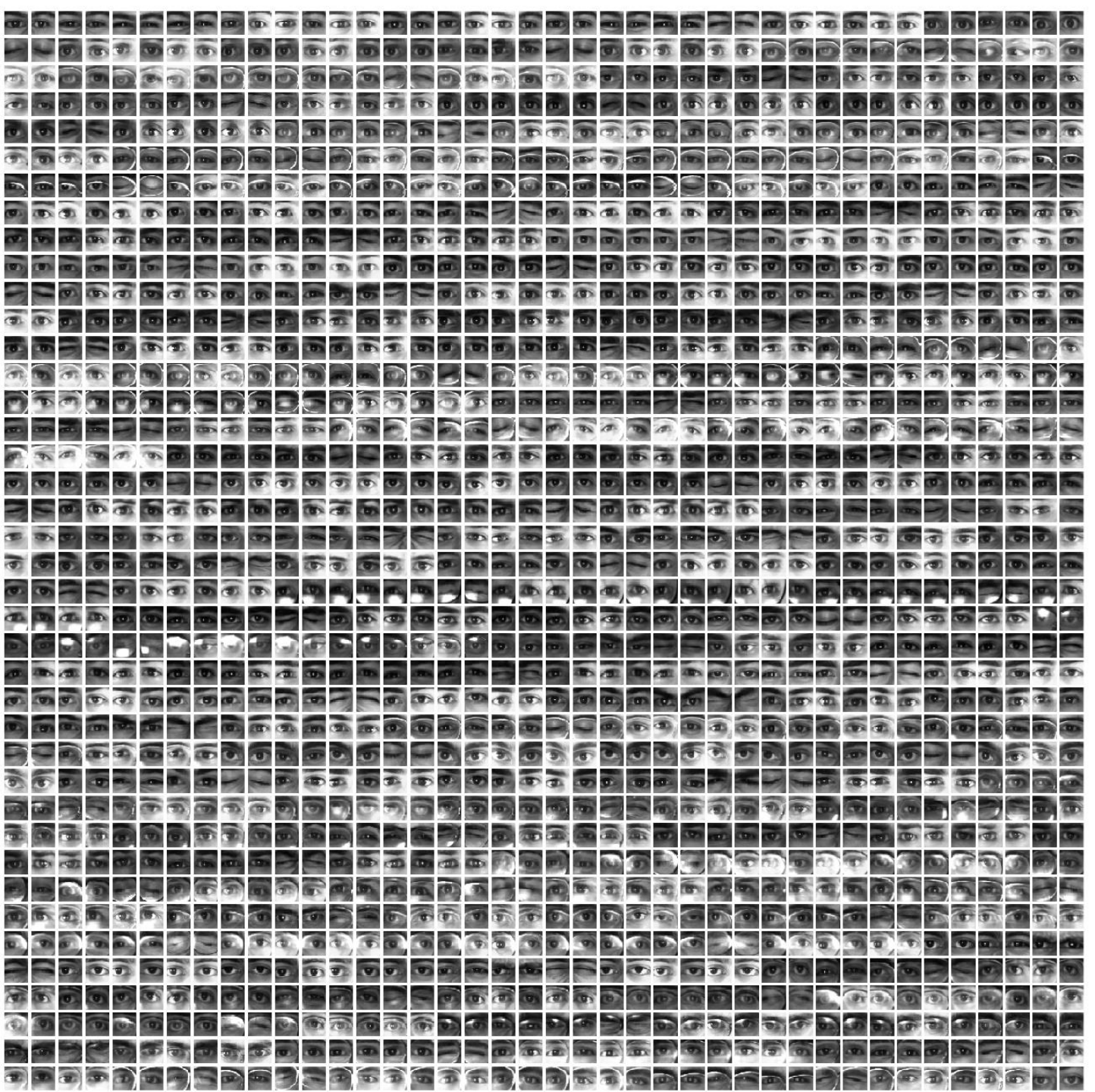

face_006_16.png
face_006_17.png
face_006_18.png
face_006_19.png


face_007_01.png
face_007_02.png
face_007_03.png
face_007_04.png

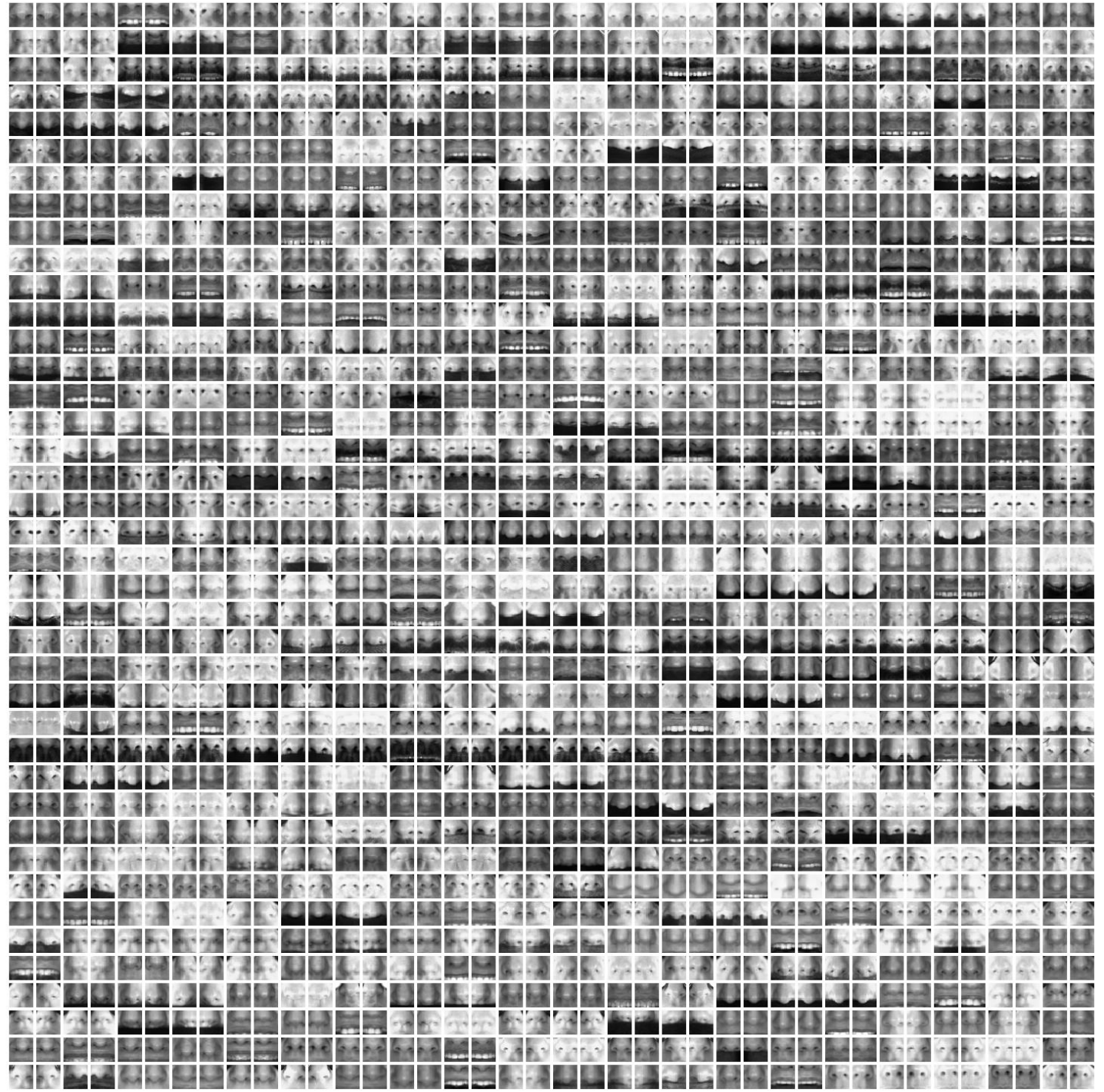

face_009_04.png
face_009_05.png
face_009_06.png
face_009_07.png


face_009_15.png
face_009_16.png
face_009_17.png
face_009_18.png

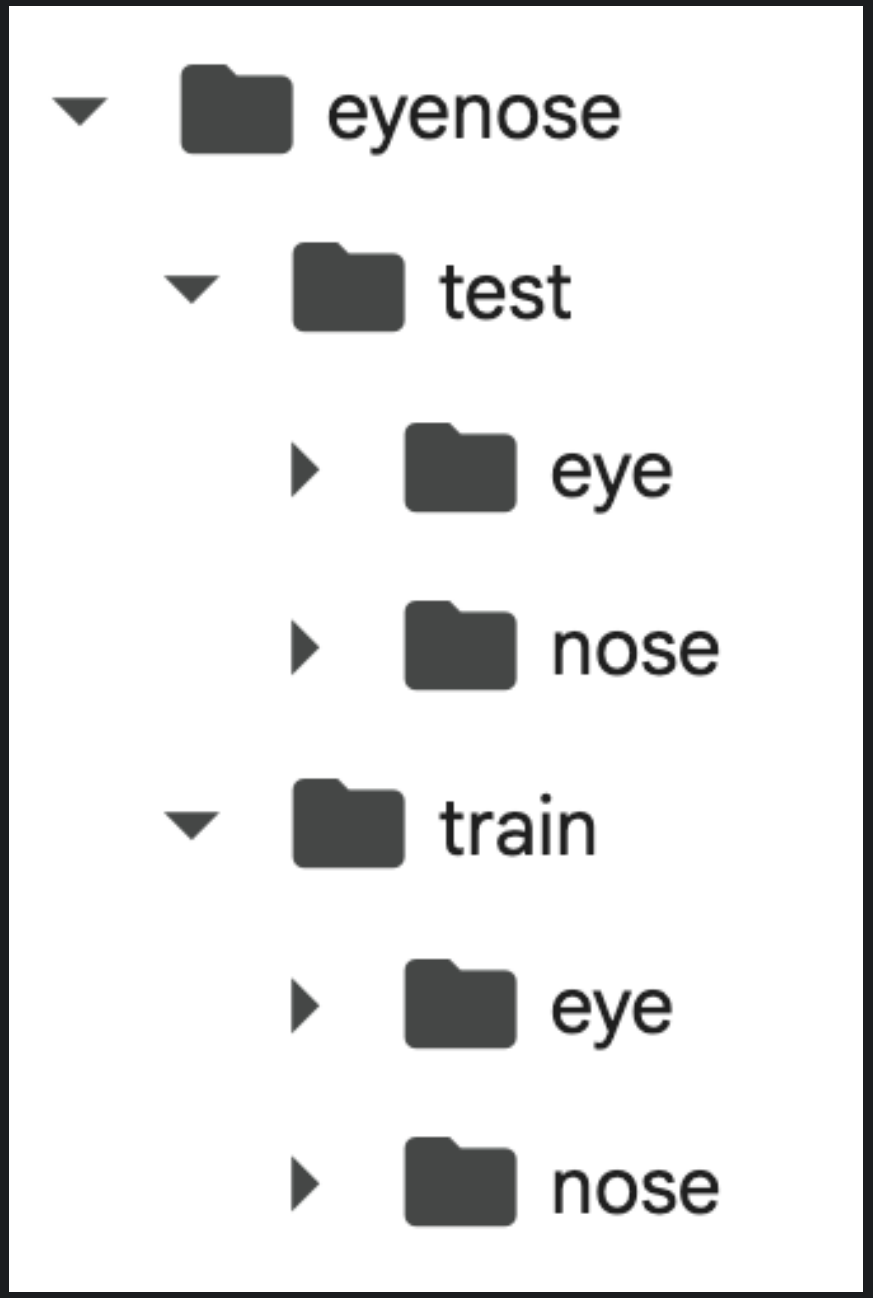
Eyes



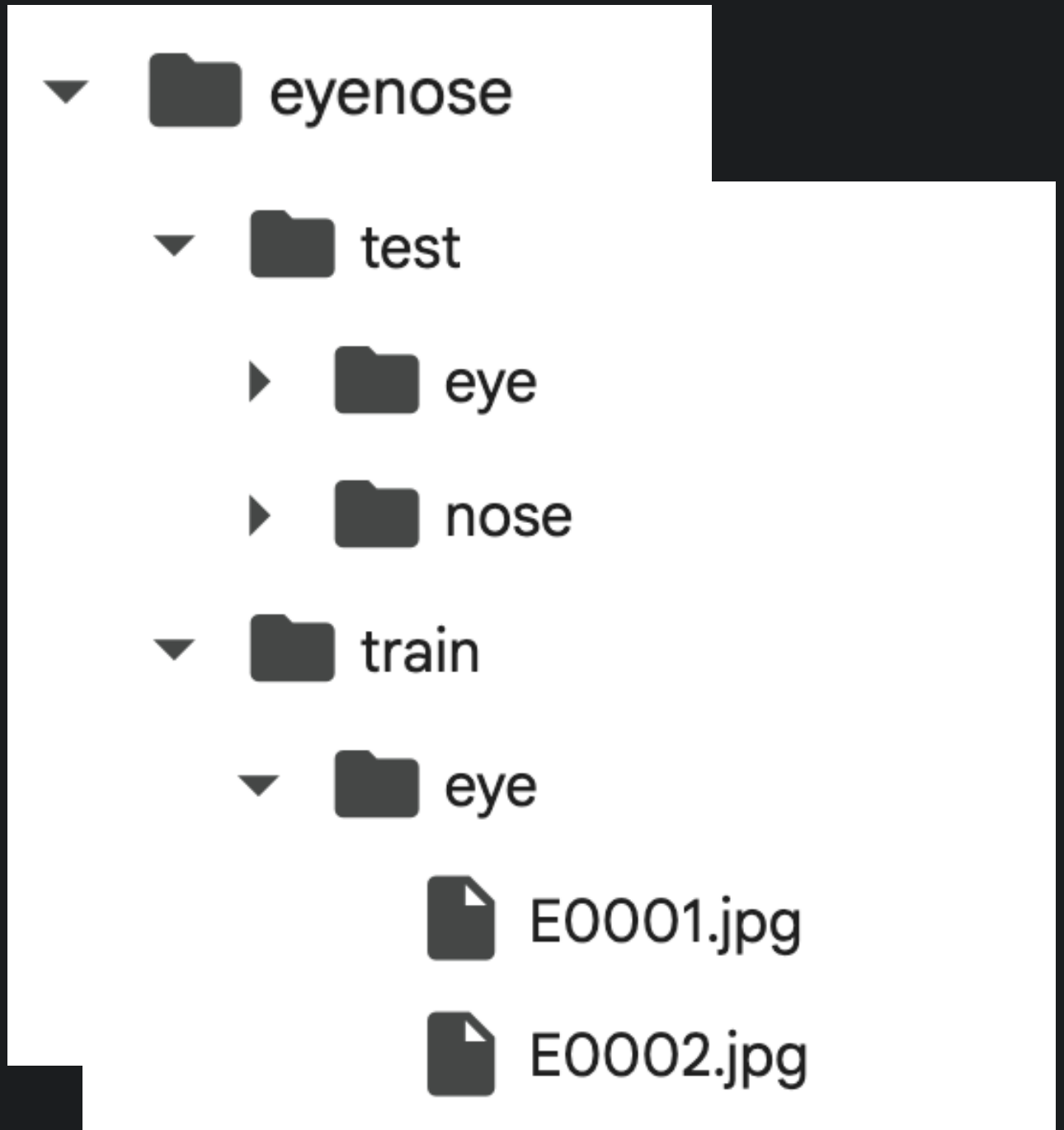
Noses



Eyes & Noses



Eyes & Noses



Eyes
&
No

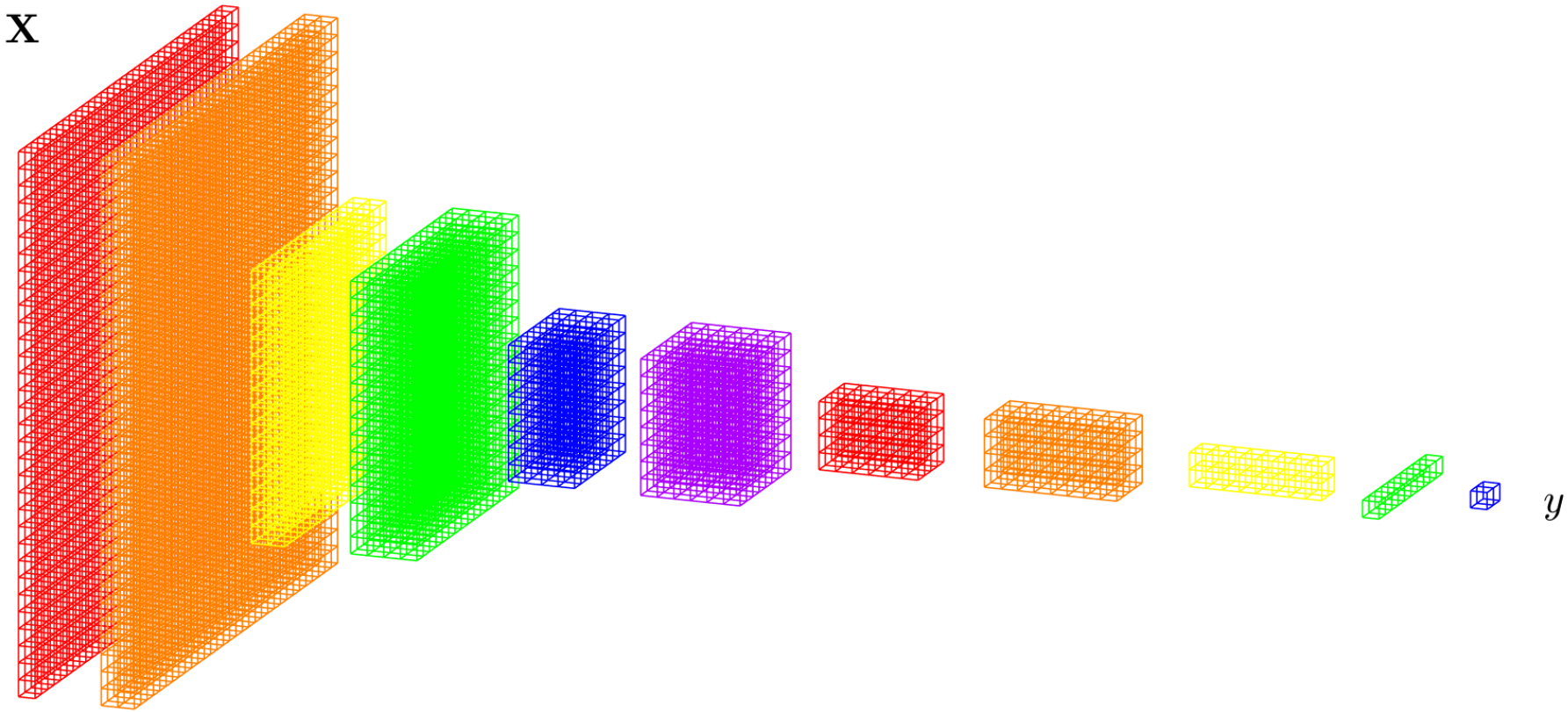
```
import torch
import torchvision
from   torchvision import transforms
from   torchvision.datasets import ImageFolder
from   torch.utils.data.dataloader import DataLoader
from   torch.utils.data import random_split
from   sklearn.metrics import confusion_matrix, accuracy_score

#train and test data directory
train_dir = "eyenose/train"
test_dir  = "eyenose/test"

#load the train and test data
train_set = ImageFolder(train_dir,transform = transforms.Compose([
    transforms.Resize((32,32)),transforms.ToTensor()]))

test_set = ImageFolder(test_dir,transforms.Compose([
    transforms.Resize((32,32)),transforms.ToTensor()]))

img,_ = train_set[0]
print('Size image: '+ str(img.shape))
print('   Classes: ',train_set.classes)
```


x
y

Implementation in PyTorch

```
class CNN_Classification(ImageClassificationBase):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(

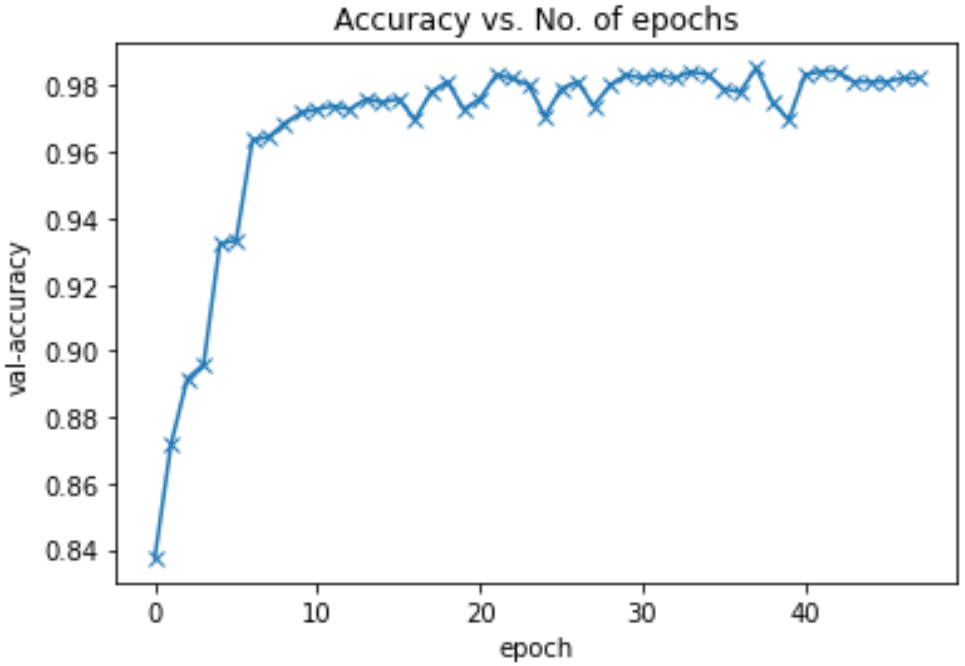
            nn.Conv2d(in_channels=3, out_channels=4, kernel_size = 5, stride = 1, padding = 0),
            nn.ReLU(),
            nn.Conv2d(in_channels=4, out_channels=8, kernel_size = 5, stride = 6, padding = 0),
            nn.ReLU(),
            nn.MaxPool2d(2,2),

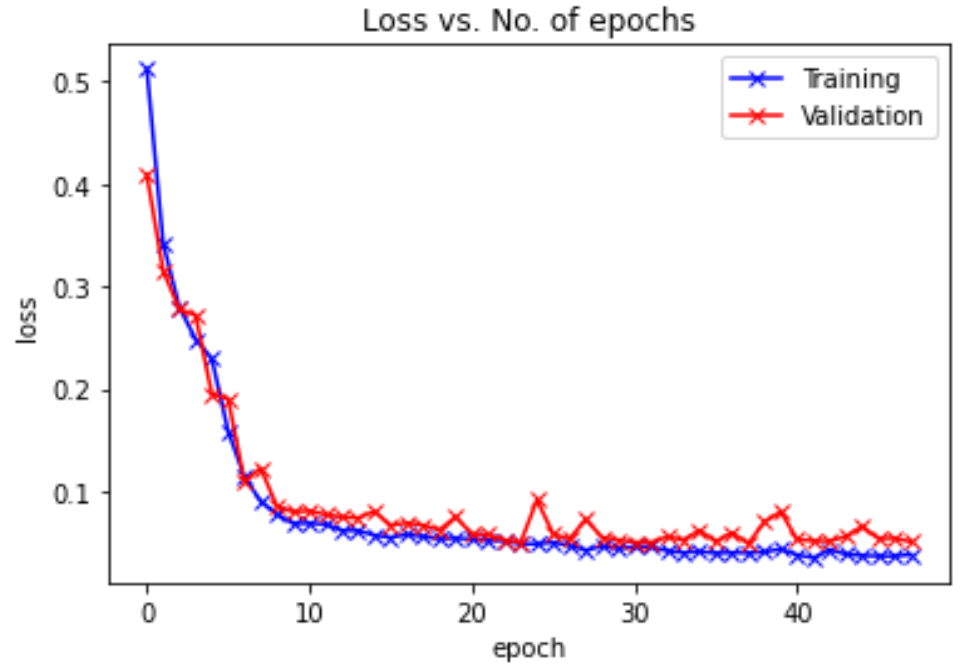
            nn.Flatten(),
            nn.Linear(32,2)
        )
```

```
CNN_Classification(
  (network): Sequential(
    (0): Conv2d(3, 4, kernel_size=(5, 5), stride=(1, 1))
    (1): ReLU()
    (2): Conv2d(4, 8, kernel_size=(5, 5), stride=(6, 6))
    (3): ReLU()
    (4): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
    (5): Flatten(start_dim=1, end_dim=-1)
    (6): Linear(in_features=32, out_features=2, bias=True)
  )
)
```

Epoch	Train-Loss	Val-Loss	Val-Acc	Best	Time [sec]
0	0.5126	0.4093	0.8375	***	1.8
1	0.3427	0.3156	0.8719	***	1.8
2	0.2797	0.2781	0.8917	***	1.8
3	0.2486	0.2719	0.8958	***	1.8
4	0.2295	0.1950	0.9323	***	1.8
5	0.1588	0.1904	0.9333	***	1.8
6	0.1145	0.1096	0.9635	***	1.8
7	0.0904	0.1229	0.9646	***	1.8
8	0.0776	0.0855	0.9687	***	1.8
9	0.0688	0.0801	0.9719	***	1.8
10	0.0689	0.0811	0.9729	***	1.8
11	0.0681	0.0774	0.9740	***	1.8
12	0.0620	0.0750	0.9729		1.8
13	0.0617	0.0742	0.9760	***	1.8
14	0.0572	0.0806	0.9750		1.8
15	0.0551	0.0670	0.9760		1.8
16	0.0580	0.0692	0.9698		1.8
17	0.0560	0.0671	0.9781	***	1.8
18	0.0538	0.0624	0.9813	***	1.8
19	0.0544	0.0752	0.9729		1.8
20	0.0533	0.0582	0.9760		1.8
21	0.0522	0.0578	0.9833	***	1.9
22	0.0519	0.0526	0.9823		1.8
23	0.0489	0.0503	0.9802		1.8
24	0.0488	0.0918	0.9708		1.8
25	0.0504	0.0582	0.9792		1.8
26	0.0470	0.0535	0.9813		1.8
27	0.0428	0.0728	0.9740		1.8
28	0.0463	0.0551	0.9802		1.8
29	0.0443	0.0527	0.9833	***	1.8
30	0.0459	0.0504	0.9823		1.8
31	0.0461	0.0501	0.9833		1.8
32	0.0420	0.0557	0.9823		1.8
33	0.0402	0.0528	0.9844	***	1.8
34	0.0414	0.0619	0.9833		1.8
35	0.0397	0.0510	0.9792		1.9
36	0.0403	0.0597	0.9781		1.8
37	0.0400	0.0491	0.9854	***	1.8
38	0.0410	0.0704	0.9750		1.8
39	0.0438	0.0802	0.9698		1.8
40	0.0382	0.0532	0.9833		1.9
41	0.0350	0.0517	0.9844		1.8
42	0.0432	0.0516	0.9844		1.8
43	0.0387	0.0561	0.9813		1.8
44	0.0374	0.0660	0.9813		1.8
45	0.0370	0.0540	0.9813		1.8
46	0.0369	0.0544	0.9823		1.8
47	0.0383	0.0509	0.9823		1.8

```
*** Early stop after 10 epochs with no improvement
Best model saved best_model.pt (val_acc = 0.9854 in epoch =  37)
Last model saved last_model.pt (val_acc = 0.9823 in epoch =  47)
Training Time: 85.82 sec
```





Epoch	Train-Loss	Val-Loss	Val-Acc	Best	Time [sec]
0	0.5126	0.4093	0.8375	***	1.8
1	0.3427	0.3156	0.8719	***	1.8
2	0.2797	0.2781	0.8917	***	1.8
3	0.2486	0.2719	0.8958	***	1.8
4	0.2295	0.1950	0.9323	***	1.8
5	0.1588	0.1904	0.9333	***	1.8
6	0.1145	0.1096	0.9635	***	1.8
7	0.0904	0.1229	0.9646	***	1.8
8	0.0776	0.0855	0.9687	***	1.8
9	0.0688	0.0801	0.9719	***	1.8
10	0.0689	0.0811	0.9729	***	1.8
11	0.0681	0.0774	0.9740	***	1.8
12	0.0620	0.0750	0.9729		1.8
13	0.0617	0.0742	0.9760	***	1.8
14	0.0572	0.0806	0.9750		1.8
15	0.0551	0.0670	0.9760		1.8
16	0.0580	0.0692	0.9698		1.8
17	0.0560	0.0671	0.9781	***	1.8
18	0.0538	0.0624	0.9813	***	1.8
19	0.0544	0.0752	0.9729		1.8
20	0.0533	0.0582	0.9760		1.8
21	0.0522	0.0578	0.9833	***	1.9
22	0.0519	0.0526	0.9823		1.8
23	0.0489	0.0503	0.9802		1.8
24	0.0488	0.0918	0.9708		1.8
25	0.0504	0.0582	0.9792		1.8
26	0.0470	0.0535	0.9813		1.8
27	0.0428	0.0728	0.9740		1.8
28	0.0463	0.0551	0.9802		1.8
29	0.0443	0.0527	0.9833	***	1.8
30	0.0459	0.0504	0.9823		1.8
31	0.0461	0.0501	0.9833		1.8
32	0.0420	0.0557	0.9823		1.8
33	0.0402	0.0528	0.9844	***	1.8
34	0.0414	0.0619	0.9833		1.8
35	0.0397	0.0510	0.9792		1.9
36	0.0403	0.0597	0.9781		1.8
37	0.0400	0.0491	0.9854	***	1.8
38	0.0410	0.0704	0.9750		1.8
39	0.0438	0.0802	0.9698		1.8
40	0.0382	0.0532	0.9833		1.9
41	0.0350	0.0517	0.9844		1.8
42	0.0432	0.0516	0.9844		1.8
43	0.0387	0.0561	0.9813		1.8
44	0.0374	0.0660	0.9813		1.8
45	0.0370	0.0540	0.9813		1.8
46	0.0369	0.0544	0.9823		1.8
47	0.0383	0.0509	0.9823		1.8

*** Early stop after 10 epochs with no improvement

Best model saved best_model.pt (val_acc = 0.9854 in epoch = 37)
Last model saved last_model.pt (val_acc = 0.9823 in epoch = 47)

Training Time: 85.82 sec

```
Training Confusion Matrix =
[[3616   45]
 [  36 3663]]

Training Accuracy = 0.9890

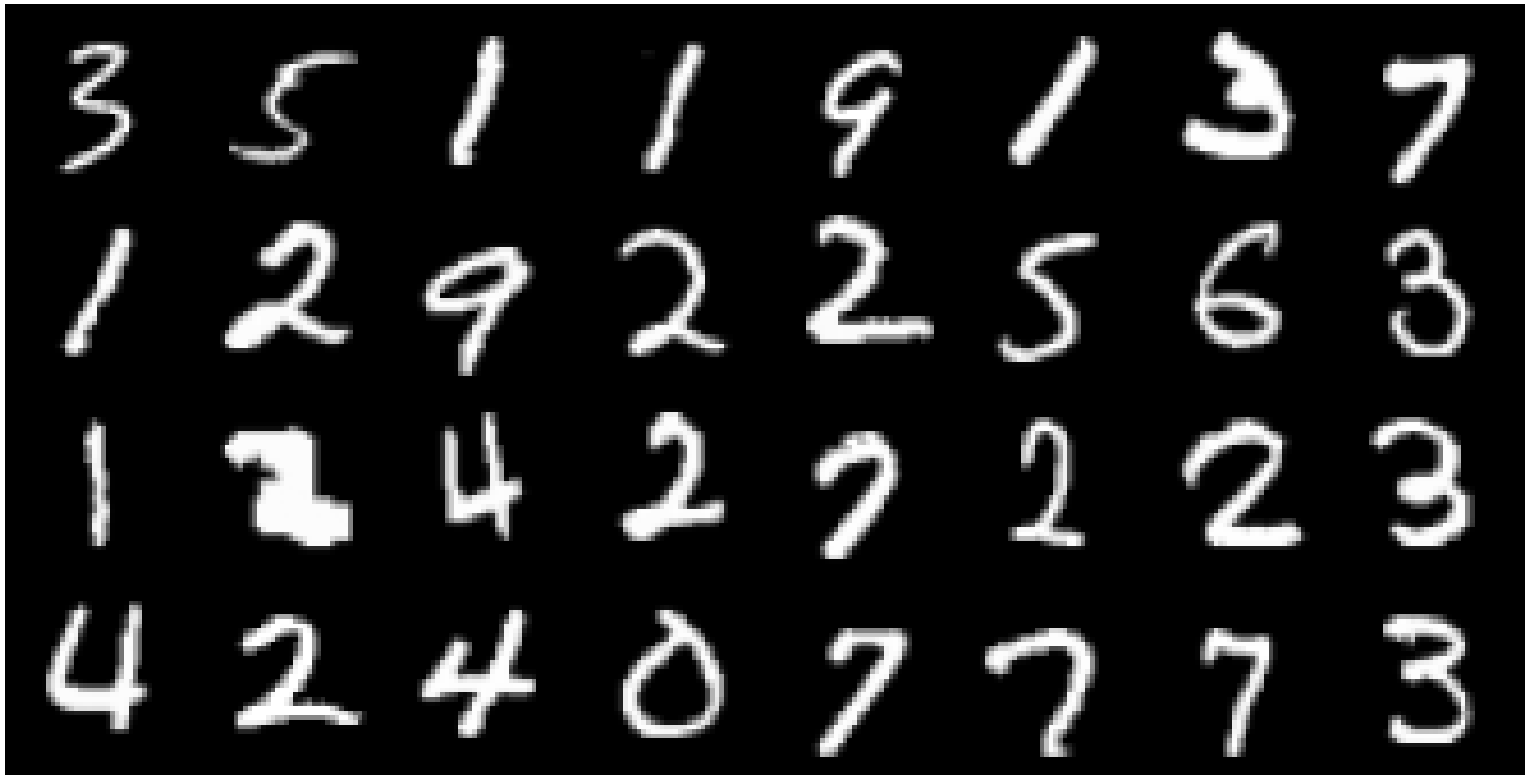
Validation Confusion Matrix =
[[490   9]
 [  5 456]]

Validation Accuracy = 0.9854

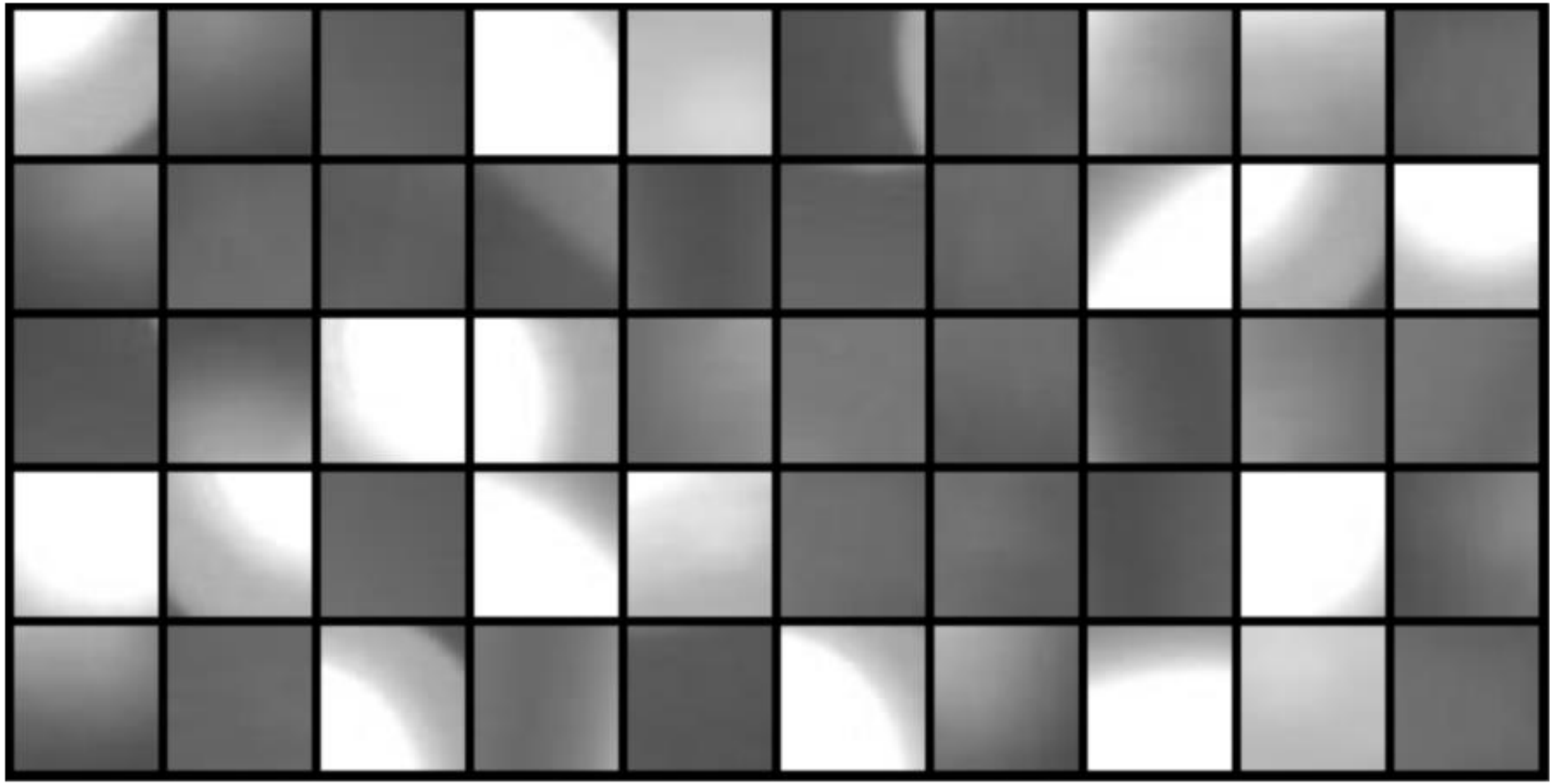
Testing Confusion Matrix =
[[1027   13]
 [   6 1034]]

Testing Accuracy = 0.9909
```

More Examples: MNIST



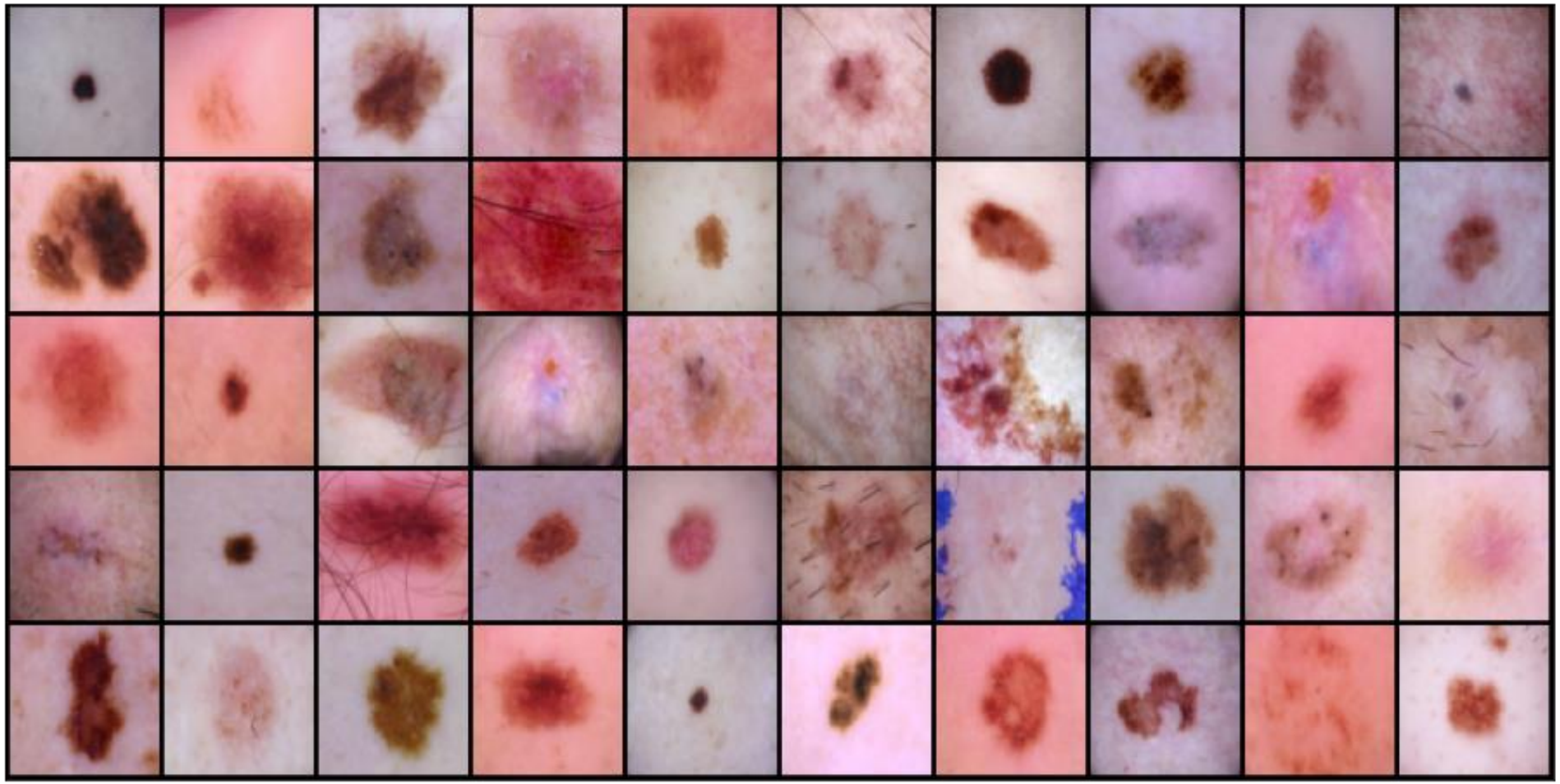
More Examples: Defects



More Examples: Cats vs Dogs



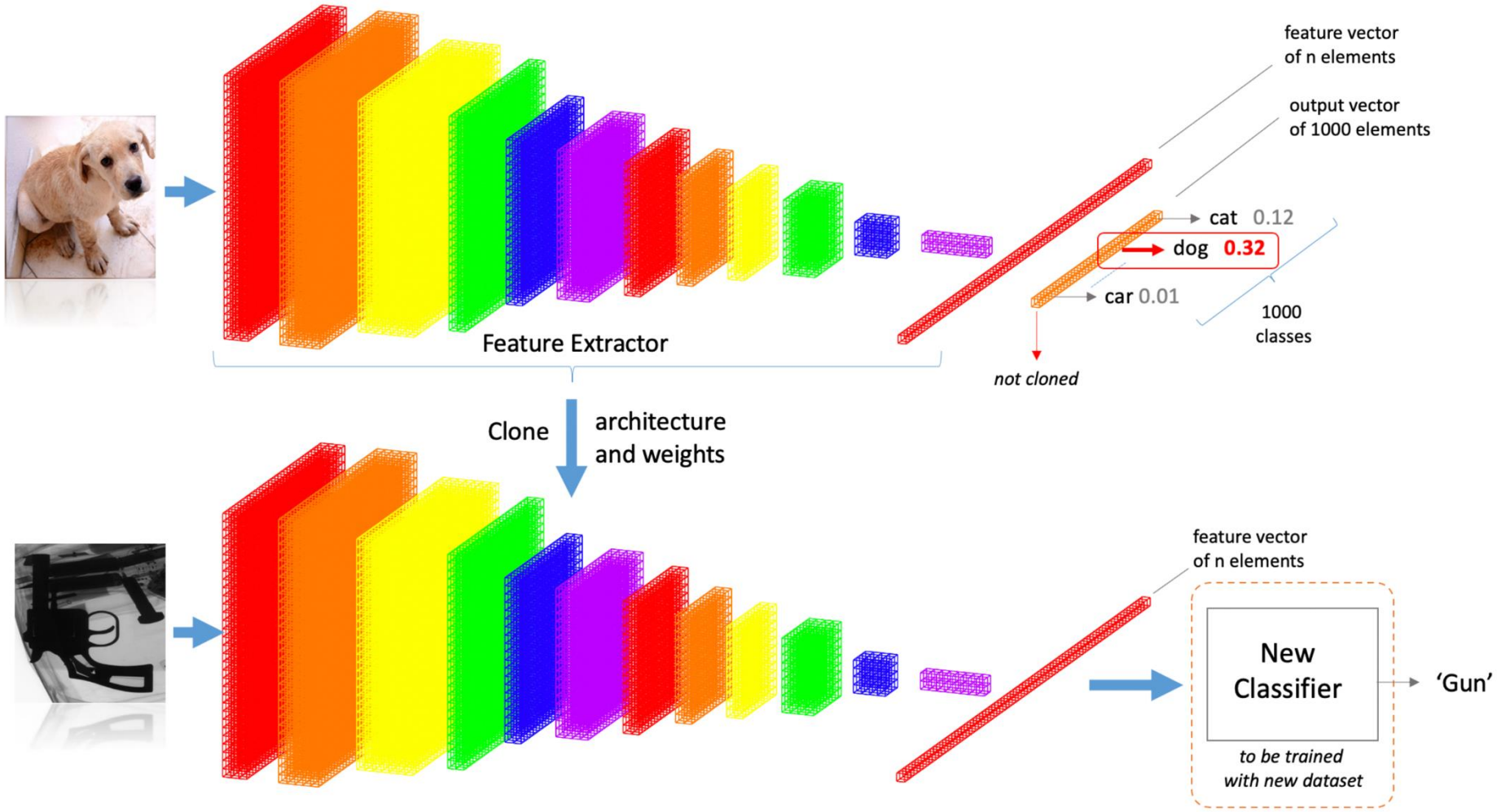
More Examples: Skin Cancer 2

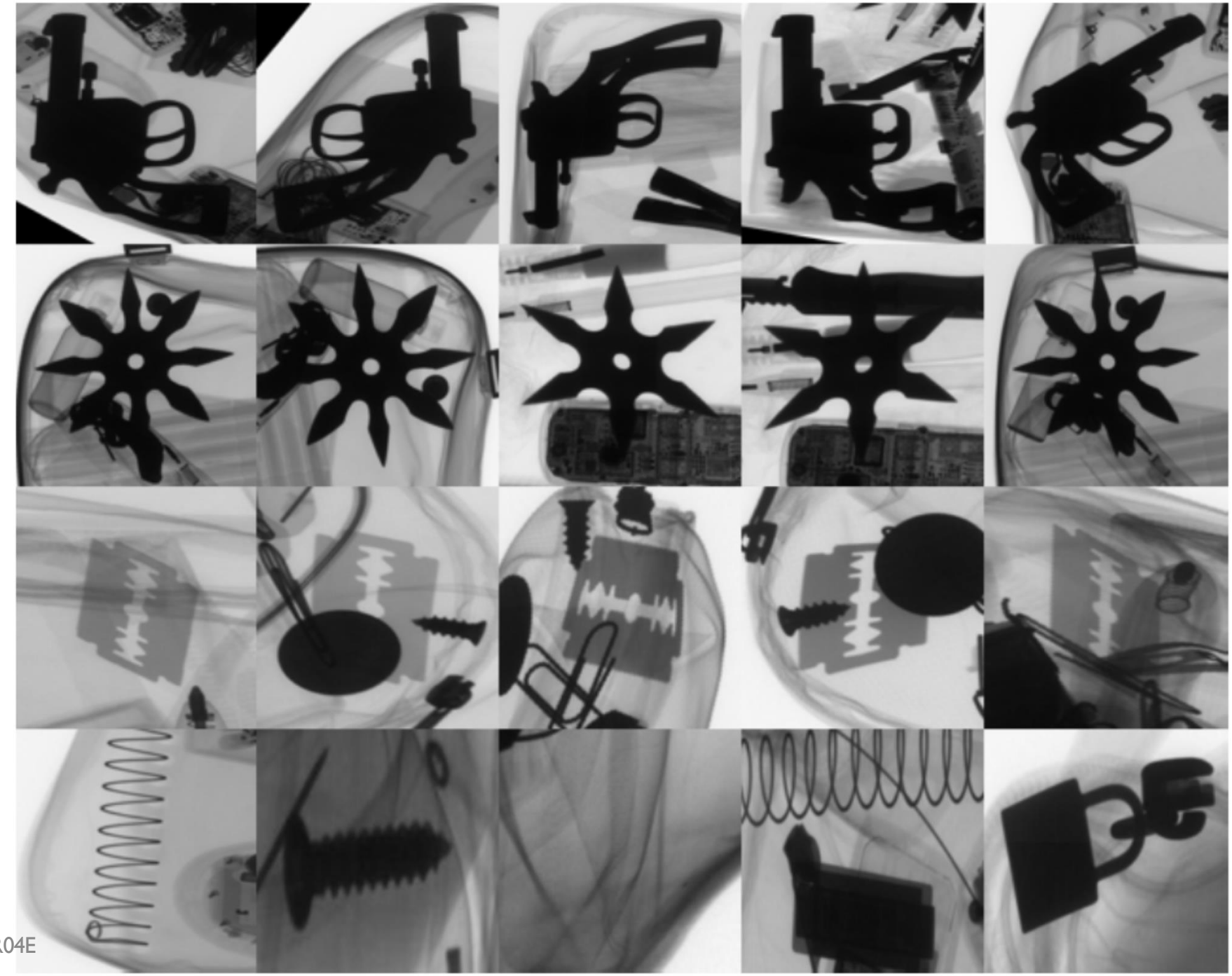


More Examples: Skin Cancer 7



Pre-Trained Models





Method	Implementation	Features	Gun		Shuriken		Blade		Total
			Pr	Re	Pr	Re	Pr	Re	η
ResNet50	2	2048	1.00	0.90	1.00	1.00	0.35	0.55	0.80
VGG16	1	1000	0.95	0.86	0.56	1.00	0.36	0.84	0.73
VGG19	2	4096	0.78	0.72	0.27	1.00	0.18	0.58	0.53
AlexNet	2	4096	0.80	0.96	0.84	1.00	0.62	0.54	0.79
DenseNet121	1	1000	0.99	0.82	0.99	0.98	0.34	0.61	0.78
GoogleNet	2	1024	0.89	0.98	0.74	1.00	0.23	0.20	0.67
InceptionResNetV2	0	1536	0.90	0.60	0.82	0.99	0.55	0.60	0.74
InceptionV3	0	2048	0.85	0.69	0.76	1.00	0.47	0.40	0.69
MobileNet	2	1280	0.98	0.84	0.94	1.00	0.35	0.95	0.82
RCNN-ILSVRC13	2	4096	0.75	1.00	0.69	1.00	0.37	0.15	0.64
ShuffleNet	2	544	0.97	0.78	0.86	1.00	0.20	0.95	0.74
SqueezeNet	2	1000	0.96	0.38	0.27	1.00	0.06	0.35	0.42
Xception	0	2048	0.54	0.12	0.75	0.78	0.06	0.21	0.38
ZfNet512	2	1024	0.87	0.84	0.28	1.00	0.04	0.10	0.48